COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science



Ghost Interview

(AI Mock Interviewer)

Complete Project Documentation

Project Proposal, Software Requirements Specification (SRS), Software Design Document (SDD), Implementation and Software Testing

for the Degree of

Bachelor of Science in Computer Science

Submitted By

Ehsan Hamza (SP23-BCS-019)

Shaharyar Altaf (SP23-BCS-055)

Supervisor

Sir Qizar Javed

Session: Spring 2023–2027

Dedication

We dedicate this work to our parents, whose unwavering support, sacrifices, and prayers have been the foundation of our success. Their encouragement, patience, and belief in our abilities have guided us throughout this journey.

We also dedicate this to our teachers and mentors who inspired us to pursue knowledge with dedication and integrity.

Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor, Sir Qizar Javed, for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this project. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

Certificate of Approval

This is to certify that the Final Year Project titled “GHOST INTERVIEW (AI MOCK INTERVIEWER)” has been developed by Ehsan Hamza (SP23-BCS-019) and Shaharyar Altaf (SP23-BCS-055) under the supervision of Sir Qizar Javed and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science.

_____

Supervisor

_____

External Examiner

_____

Head of Department

Department of Computer Science

Plagiarism and AI Content Certificate

Date: _____

It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the [Report Type: Complete Project Documentation] submitted by:

- Ehsan Hamza (SP23-BCS-019)
- Shaharyar Altaf (SP23-BCS-055)

titled:

“GHOST INTERVIEW (AI MOCK INTERVIEWER)”

is [XX%] which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as [XX%]. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

Supervisor Name: _____

Designation: _____

Signature: _____

Department: Department of Computer Science

University: COMSATS University Islamabad, Vehari Campus

Executive Summary

AI Interview Ghost (Mock Interviewer) is an intelligent web-based interview preparation system designed to simulate real-time job interviews using artificial intelligence. The system provides users with an interactive interview experience through a virtual AI avatar that asks domain-specific interview questions using a realistic human-like voice. Unlike traditional mock interview systems, this project focuses not only on the verbal responses of the user but also on non-verbal cues such as facial expressions, eye contact, confidence level, and emotional state.

The proposed system analyzes the candidate’s speech, emotions, and body language using advanced AI models and generates a comprehensive Confidence and Competency Report. This report highlights communication skills, nervousness indicators, usage of filler words, and overall interview readiness. The system aims to help students and job seekers improve their interview performance through data-driven feedback and self-assessment.

Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
CSRF	Cross-Site Request Forgery
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
FK	Foreign Key
FR	Functional Requirement
FYP	Final Year Project
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
NLP	Natural Language Processing
PDF	Portable Document Format
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
SDD	Software Design Document
SMTP	Simple Mail Transfer Protocol
SRS	Software Requirements Specification
TC	Test Case
UC	Use Case
UI	User Interface
UML	Unified Modeling Language
UUID	Universally Unique Identifier
UX	User Experience

Table of Contents

Dedication..... i
Acknowledgement..... ii
Certificate of Approval..... iii
Plagiarism and AI Content Certificate..... iv
Executive Summary..... v
Abbreviations..... vi
List of Tables..... x
List of Figures..... xi

1 Introduction..... 1
1.1 Introduction..... 1
1.2 Project Category..... 1
1.3 Related System Analysis / Literature Review..... 1
1.4 Advantages / Benefits of the Proposed System..... 2
1.5 System Limitations / Constraints..... 2
1.6 Tools and Technologies..... 2
1.7 Chapter Summary..... 2

2 Problem Definition..... 4
2.1 Problem Statement..... 4
2.2 Proposed Solution Overview..... 4
2.3 Scope of the System..... 4
2.4 System Modules..... 5
2.5 Software Process Methodology..... 5
2.6 Project Stakeholders and Roles..... 5
2.7 Team Members Individual Tasks / Work Division..... 5
2.8 Data Gathering Approach..... 5
2.9 Concepts..... 6
2.10 Gantt Chart..... 6
2.11 Mockups..... 6
2.12 Chapter Summary..... 9

3 Requirement Analysis..... 10
3.1 Introduction to Requirement Analysis..... 10
3.2 Requirement Elicitation Techniques..... 10
3.3 User Classes and Characteristics..... 10
3.4 System Overview..... 11
3.5 Use Case Model..... 11
3.5.1 Use Case Diagram..... 11
3.5.2 Use Case Descriptions..... 12
3.5.2.1 Module 1: User Management..... 12
3.5.2.2 Module 2: AI Interview Module..... 13
3.5.2.3 Module 3: Reporting and Dashboard..... 14
3.5.3 Event-Response Table..... 15
3.6 Functional Requirements..... 15

3.6.1 Module 1: User Management....15
3.6.1.1 FR-1.1 User Registration....15
3.6.1.2 FR-1.2 User Authentication....16
3.6.2 Module 2: AI Interview Module....16
3.6.2.1 FR-2.1 Job Role Selection....16
3.6.2.2 FR-2.2 AI Avatar Interview Conduction....16
3.6.2.3 FR-2.3 Speech-to-Text Conversion....17
3.6.2.4 FR-2.4 Filler Word Detection....17
3.6.2.5 FR-2.5 Facial Emotion Detection....17
3.6.2.6 FR-2.6 Sentiment Analysis....18
3.6.3 Module 3: Reporting and Analytics....18
3.6.3.1 FR-3.1 Confidence and Competency Report Generation....18
3.6.3.2 FR-3.2 Interview History Dashboard....18
3.7 Non-Functional Requirements....19
3.7.1 Performance Requirements....19
3.7.2 Security Requirements....19
3.7.3 Reliability Requirements....19
3.7.4 Usability Requirements....19
3.7.5 Scalability Requirements....20
3.7.6 Maintainability Requirements....20
3.8 External Interface Requirements....20
3.8.1 User Interface Requirements....20
3.8.2 Hardware Interface Requirements....20
3.8.3 Software Interface Requirements....21
3.8.4 Communication Interface Requirements....21
3.9 Assumptions and Dependencies....21
3.10 Requirement Traceability Matrix....21
3.11 Chapter Summary....22

4 Software Design....23
4.1 Introduction to Software Design....23
4.2 Design Methodology and Software Process Model....23
4.2.1 Design Methodology....23
4.2.2 Software Process Model....23
4.3 System Overview....24
4.4 Architectural Design....25
4.5 Design Models....26
4.5.1 Activity Diagrams....26
4.5.1.1 Module 1: User Management....26
4.5.1.2 Module 2: AI Interview Core Module....27
4.5.1.3 Module 3: Admin Module....28
4.5.2 Class Diagram....29
4.5.3 Sequence Diagrams....30
4.5.3.1 Sequence Diagram – User Login....30
4.5.3.2 Sequence Diagram – AI Interview Core Operation....31
4.5.4 State Transition Diagrams....32
4.5.4.1 Interview Session States....32
4.5.4.2 User Account States....32
4.6 Data Flow Diagrams....33
4.6.1 Level 1 Data Flow Diagram....33

4.6.2 Level 2 Data Flow Diagram....34
4.7 Data Design....35
4.7.1 Entity Relationship Diagram (ERD)....35
4.7.2 Database Architecture....36
4.8 Database Schema Diagram....36
4.8.1 Data Dictionary....37
4.9 Database Tables / Collections....37
4.9.1 User Collection....39
4.9.2 Core Data Collections....40
4.9.3 Relationships Between Collections....40
4.9.4 Security and Data Integrity....40
4.9.5 Scalability Considerations....41
4.10 Chapter Summary....41

5 Implementation....42
5.1 Introduction....42
5.1.1 Project Objectives....42
5.1.2 Technology Stack....43
5.2 Implementation (60% Modules Completion)....43
5.2.1 User Registration Module....43
5.2.2 User Login & Authentication Module....44
5.2.3 Job Role Selection & Interview Initiation Module....45
5.2.4 AI Interview Session Module....47
5.2.5 Speech-to-Text, Filler-Word & Sentiment Analysis Module....49
5.2.6 Facial Emotion Detection Module....51
5.2.7 Confidence & Competency Report Module....52
5.2.8 Interview History Dashboard Module....54
5.2.9 Admin Panel Module....55
5.2.10 Backend REST API & Security Module....57
5.3 Module Completion Summary....58

6 Software Testing....59
6.1 Introduction....59
6.2 Authentication Module Testing....59
6.3 Interview Workflow & AI Module Testing....60
6.4 Reporting, Dashboard & API Testing....63

7 Conclusion and Future Work....66
7.1 Conclusion....66
7.2 Future Work....66
References....67

List of Tables

1.1 Related System Analysis with proposed project solution....1
1.2 Tools and Technologies for Proposed Project....2
1.3 Software Technologies for Proposed Project....2
2.1 Project Stakeholders for Proposed Project....5
2.2 Team Member Work Division for Proposed Project....5
3.1 User Classes and Characteristics....10
3.2 UC-1.1: User Registration....12
3.3 UC-1.2: User Login....12
3.4 UC-2.1: Job Role Selection and Interview Initiation....13
3.5 UC-2.2: Speech Analysis....13
3.6 UC-2.3: Emotion and Sentiment Analysis....14
3.7 UC-3.1: Confidence and Competency Report Generation....14
3.8 Event-Response Table....15
3.9 Functional Requirement – User Registration....15
3.10 Functional Requirement – User Authentication....16
3.11 Functional Requirement – Job Role Selection....16
3.12 Functional Requirement – AI Avatar Interview....16
3.13 Functional Requirement – Speech-to-Text Conversion....17
3.14 Functional Requirement – Filler Word Detection....17
3.15 Functional Requirement – Facial Emotion Detection....17
3.16 Functional Requirement – Sentiment Analysis....18
3.17 Functional Requirement – Report Generation....18
3.18 Functional Requirement – Interview History Dashboard....18
3.19 Requirement Traceability Matrix....22
4.1 Data Dictionary – User Collection....38
4.2 Data Dictionary – InterviewSession Collection....38
4.3 Data Dictionary – SpeechRecord Collection....38
4.4 Data Dictionary – EmotionRecord Collection....39
4.5 Data Dictionary – Report Collection....39
4.6 User Collection Structure....40
5.1 Technology Stack of the Implemented System....43
5.2 Module Completion Summary at Evaluation III....58
6.1 Test Cases – Authentication Module....59
6.2 Test Cases – Interview Workflow and AI Modules....61
6.3 Test Cases – Reporting, Dashboard and API....63

List of Figures

2.1 Gantt Chart....6
2.2 Job Role Selection Screen....7
2.3 AI Interview Avatar Interaction Screen....7
2.4 Emotion and Speech Analysis Dashboard....8
2.5 Confidence and Competency Report Screen....8
3.1 Use Case Diagram of Ghost Interview System....11
4.1 Context Diagram of Ghost Interview System....25
4.2 System Architecture Diagram of Ghost Interview....26
4.3 Activity Diagram – User Management Module....27
4.4 Activity Diagram – AI Interview Core Module....28
4.5 Activity Diagram – Admin Module....29
4.6 Class Diagram of Ghost Interview System....30
4.7 Sequence Diagram – User Login....31
4.8 Sequence Diagram – AI Interview Core Operation....32
4.9 State Transition Diagram – Interview Session....33
4.10 Level 1 Data Flow Diagram – Ghost Interview System....34
4.11 Level 2 Data Flow Diagram – AI Interview Engine Module....35
4.12 Entity Relationship Diagram – Ghost Interview System....36
4.13 Database Schema Diagram – Ghost Interview System....37
5.1 Registration form with name, e-mail and password fields....44
5.2 Login interface with e-mail and password fields....45
5.3 Job-role selection screen with available roles....46
5.4 Interview initiated — first question presented to the candidate....47
5.5 Live interview screen with AI avatar, question text and webcam feed....48
5.6 Live speech-to-text transcript of the candidate's spoken answer....48
5.7 Browser-side emotion detection running over the webcam feed....49
5.8 Per-answer analysis showing filler-word count and sentiment....50
5.9 Filler words detected across a full interview session....50
5.10 Real-time facial expression detection during the interview....51
5.11 Batched emotion readings recorded for the session....52
5.12 Confidence & Competency Report with overall readiness ring....53
5.13 Performance breakdown — confidence, competency, communication and stability....53
5.14 Emotion distribution and strengths / improvement areas....54
5.15 One-click PDF export of the report....54
5.16 Dashboard showing interview history and past reports....55
5.17 Admin user-management panel with search and status controls....56
5.18 Admin job-role management screen....57
5.19 Platform-wide analytics overview....57
6.1 TC-L1 — successful login and redirection....60
6.2 TC-I1 — interview session started with first question....61

6.3 TC-I2 — answer submitted with per-answer analysis.....62
6.4 TC-I3 — emotion batch recorded during the session.....62
6.5 TC-I4 — report generated after completing the session.....63
6.6 TC-P1 — Confidence & Competency Report with PDF export.....64
6.7 TC-D1 — interview history dashboard.....64
6.8 TC-A1 — admin analytics overview.....65

Chapter 1

Introduction

1.1 Introduction

This document presents the project scope for the proposed system titled AI Interview Ghost (Mock Interviewer). The purpose of this project is to provide an intelligent and interactive interview preparation platform using artificial intelligence technologies. Interviews are a critical part of job selection processes, and many candidates struggle due to lack of practice and real-time feedback. Traditional mock interviews are limited in availability and often lack detailed performance analysis.

With the advancement of AI, it is now possible to simulate realistic interview scenarios. This system integrates voice interaction, facial emotion detection, and speech analysis to create a virtual interviewer experience. The AI avatar interacts with users through voice-based questions and evaluates their responses using multiple AI models. The project is developed as a web-based application to ensure accessibility and scalability. The system is particularly useful for students, fresh graduates, and professionals preparing for technical or HR interviews.

1.2 Project Category

Selected domain of the proposed project: **B – Web Application / Web Application based Information System**.

1.3 Related System Analysis / Literature Review

System 1: HireVue

HireVue provides AI-based video interviews but lacks detailed emotional feedback and real-time guidance.

System 2: Pramp

Pramp offers peer-to-peer mock interviews but depends on human availability and lacks AI-based emotion analysis.

System 3: Interviewing.io

Interviewing.io focuses on technical interviews but does not analyze facial expressions or emotional behavior.

Table 1.1: Related System Analysis with proposed project solution

Application Name	Weakness	Proposed Project Solution
HireVue	Limited emotion insights	Deep emotion & confidence analysis
Pramp	Human dependency	Fully automated AI interviewer
Interviewing.io	No emotion detection	Facial and sentiment analysis

1.4 Advantages / Benefits of the Proposed System

- Real-time AI-based mock interview experience
- Emotion and confidence analysis using webcam
- Detection of filler words and speech patterns
- Detailed performance report generation
- Accessible web-based system
- Cost-effective alternative to human interviews
- Improves communication and interview confidence

1.5 System Limitations / Constraints

- Requires stable internet connection
- Emotion detection accuracy depends on camera quality
- Limited to predefined job roles
- Privacy concerns related to webcam usage

1.6 Tools and Technologies

Table 1.2: Tools and Technologies for Proposed Project

Tools	Version	Rationale
Visual Studio Code	Latest	Development IDE
React.js	18+	Frontend framework
Node.js	20+	Backend runtime
WebRTC	Latest	Camera & mic access

Table 1.3: Software Technologies for Proposed Project

Technology	Version	Rationale
JavaScript	ES6+	Programming
OpenAI Whisper	Latest	Speech-to-text
Gemini API	Latest	Interview logic
DeepFace / Hume AI	Latest	Emotion detection
ElevenLabs	Latest	AI voice generation

1.7 Chapter Summary

This chapter introduced Ghost Interview (AI Mock Interviewer), its project category, a comparison with existing platforms such as HireVue, Pramp and Interviewing.io, the benefits

and limitations of the proposed system, and the tools and technologies selected for its development.

Chapter 2

Problem Definition

2.1 Problem Statement

Many job candidates face difficulty during interviews due to nervousness, lack of confidence, and poor communication skills. Most available interview preparation platforms focus only on question-answer practice and ignore non-verbal communication aspects such as facial expressions, eye contact, and emotional stability. Human mock interviews are costly, time-limited, and not always available to everyone.

Existing systems also fail to provide real-time emotion analysis and speech behavior insights such as filler word usage. Candidates do not receive objective feedback on their confidence and emotional performance. There is a lack of intelligent systems that combine voice interaction, emotion detection, and AI-driven evaluation in a single platform. This project is developed to overcome these limitations and to provide a comprehensive interview preparation solution while enhancing the students' skills in artificial intelligence, machine learning, and web technologies.

2.2 Proposed Solution Overview

The proposed AI Interview Ghost system addresses the identified problems by providing a fully automated mock interview environment. The system allows users to select a job role, after which an AI avatar conducts an interview using voice-based interaction. The user responds through a webcam and microphone, enabling the system to analyze both verbal and non-verbal behavior.

Speech-to-text conversion is used to analyze spoken responses and detect filler words. Emotion detection models analyze facial expressions and eye movement to determine nervousness and confidence levels. Sentiment analysis is applied to evaluate the emotional tone of the answers. Based on these analyses, the system generates a detailed Confidence and Competency Report. This solution provides users with actionable insights and helps them improve interview performance through repeated practice.

2.3 Scope of the System

The scope of AI Interview Ghost includes developing a web-based interview simulation system using AI technologies. The system will allow users to select job roles and participate in AI-driven interviews. The AI avatar will ask interview questions using voice interaction. The system will analyze user responses through speech-to-text processing and facial emotion detection. A confidence and competency report will be generated after each interview session.

The project includes sentiment analysis, filler word detection, and emotion recognition. The system will not provide real job placements or human interview scheduling. Advanced AI model training is out of scope and pre-trained models will be used. The focus remains on interview preparation and performance analysis.

2.4 System Modules

Module 1: User Role Selection Module – Allows users to select job roles and interview categories.

Module 2: AI Interview Avatar Module – Conducts interviews using voice-based AI avatar.

Module 3: Speech Analysis Module – Converts speech to text and detects filler words.

Module 4: Emotion & Sentiment Analysis Module – Analyzes facial expressions and emotional tone.

Module 5: Confidence & Competency Report Module – Generates performance reports with insights.

Module 6: System Dashboard Module – Displays interview history and analytics.

2.5 Software Process Methodology

The Object-Oriented Software Development Methodology will be used for this project. This methodology is selected due to its modular structure and reusability. It supports scalability and effective system maintenance. It is suitable for AI-based web applications.

2.6 Project Stakeholders and Roles

Table 2.1: Project Stakeholders for Proposed Project

Project Sponsor	COMSATS University, Islamabad
Stakeholders	**Students:** System development **Supervisor:** Project guidance **FYP Committee:** Evaluation

2.7 Team Members Individual Tasks / Work Division

Table 2.2: Team Member Work Division for Proposed Project

Student Name	Registration No	Responsibility
Ehsan Hamza	SP23-BCS-019	AI models & backend
Shaharyar Altaf	SP23-BCS-055	Frontend & UI

2.8 Data Gathering Approach

Data will be gathered through interviews, online research, and analysis of existing interview platforms. Questionnaires will be used to understand user requirements. Research papers and AI documentation will also be reviewed.

2.9 Concepts

Concept 1: Artificial Intelligence – Used for interview logic and evaluation.

Concept 2: Emotion Detection – Facial expression analysis using AI models.

Concept 3: Speech-to-Text Processing – Converts user speech into text for analysis.

Concept 4: Sentiment Analysis – Evaluates emotional tone of responses.

2.10 Gantt Chart

The Gantt chart will be created using MS Project to define timelines, dependencies, and milestones for each module.

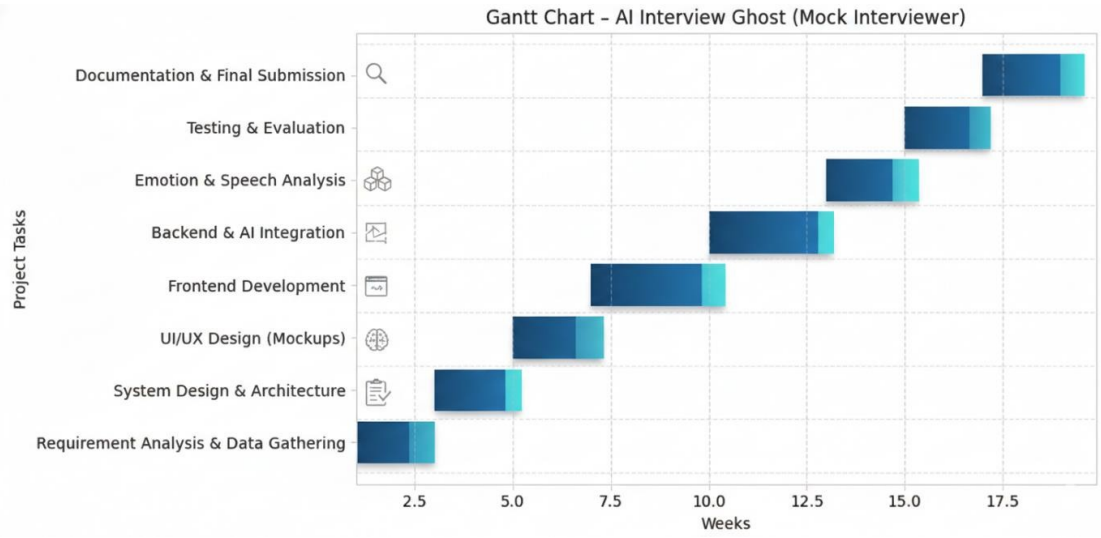


Figure 2.1: Gantt Chart

2.11 Mockups

Mockups will include interview screen, AI avatar interaction, report dashboard, and analytics view.



Figure 2.2: Job Role Selection Screen

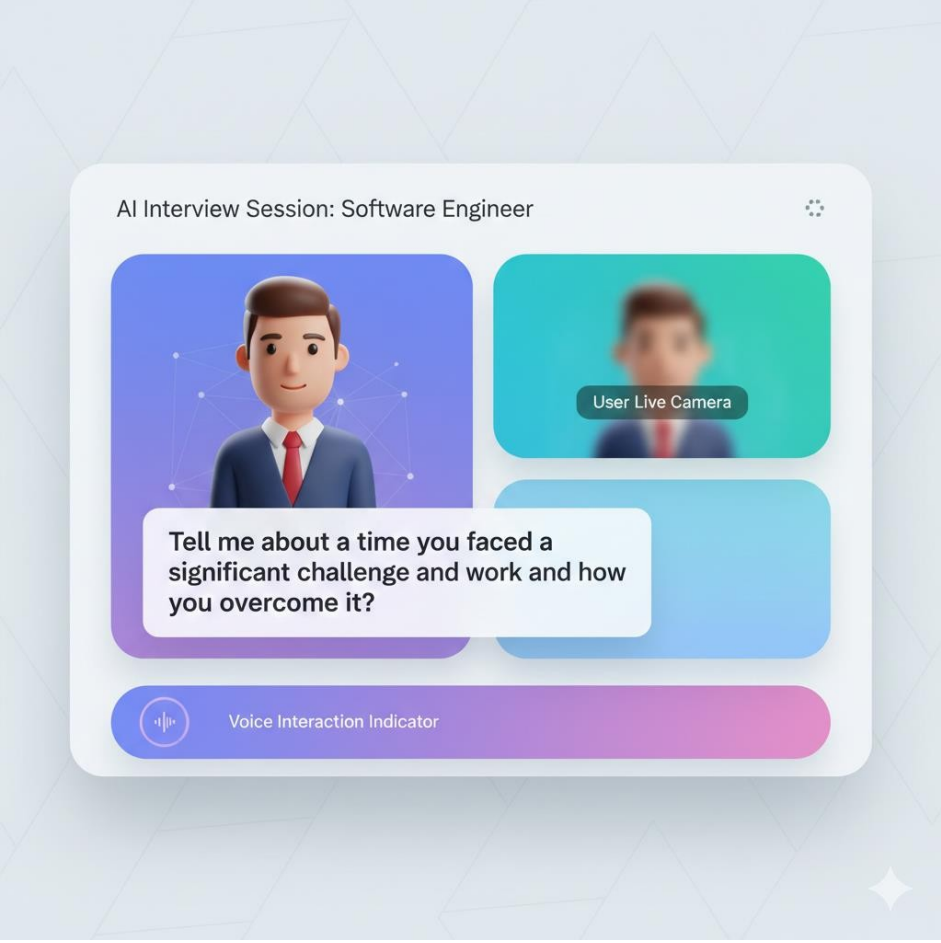


Figure 2.3: AI Interview Avatar Interaction Screen



Figure 2.4: Emotion and Speech Analysis Dashboard

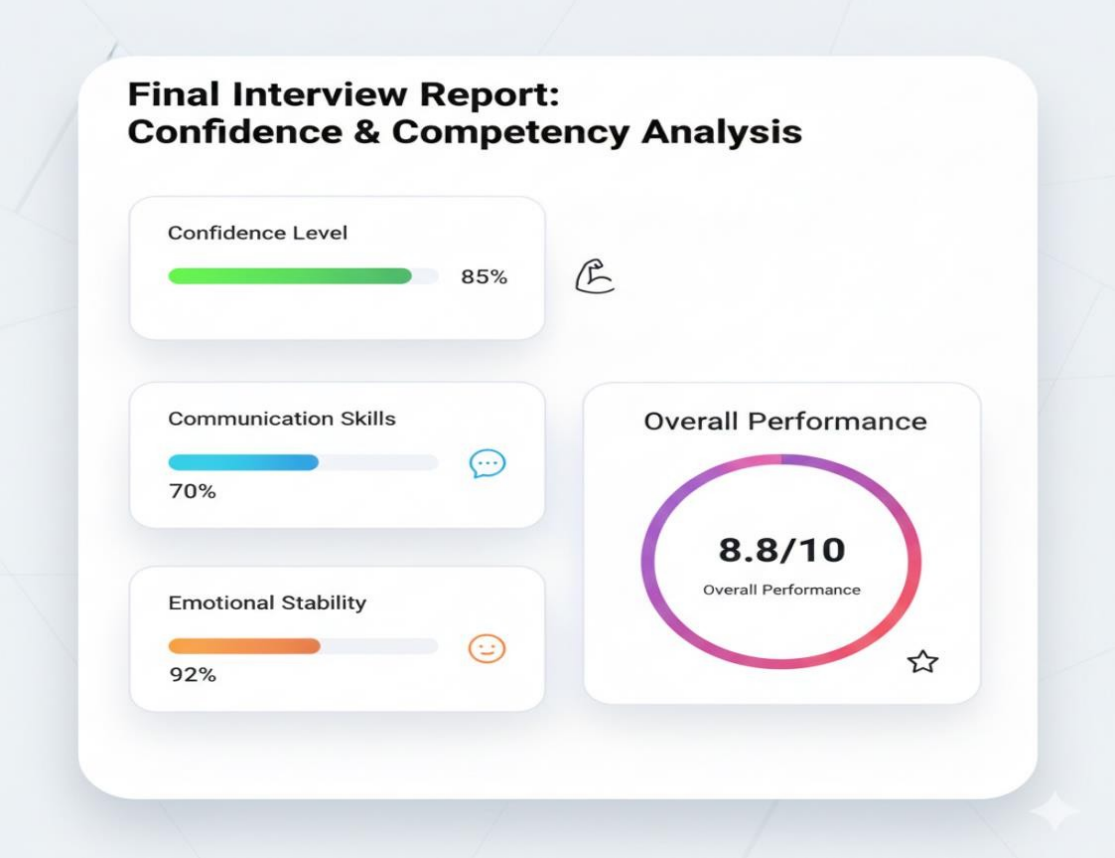


Figure 2.5: Confidence and Competency Report Screen

2.12 Chapter Summary

AI Interview Ghost is an innovative interview preparation system that combines AI-driven voice interaction, emotion analysis, and performance evaluation. The project provides a realistic interview environment and actionable feedback. It enhances user confidence and communication skills. The system demonstrates the practical application of artificial intelligence in real-world scenarios.

Chapter 3

Requirement Analysis

3.1 Introduction to Requirement Analysis

This chapter presents a structured and formal specification of system requirements for Ghost Interview AI Mock Interviewer. The purpose of this chapter is to clearly define what the system shall accomplish before proceeding to the system design phase. The requirements documented here serve as a contractual foundation for implementation, validation, and system testing.

The requirement analysis includes requirement elicitation techniques, identification of stakeholders, use case modeling, functional requirements, non-functional requirements, and external interface requirements. All requirements are written in measurable and testable format using standard requirement engineering principles.

3.2 Requirement Elicitation Techniques

Requirements for this project were identified using the following techniques:

- **Stakeholder Interviews:** To capture user expectations and constraints from students, job seekers, and supervisors.
- **Literature Review:** Analysis of research papers on emotion detection, speech analysis, and interview preparation systems.
- **Analysis of Existing Solutions:** Examination of HireVue, Pramp, and Interviewing.io to identify functional gaps.
- **Online Questionnaires:** To understand user requirements and pain points regarding interview preparation.
- **AI Documentation Review:** Study of OpenAI Whisper, Gemini API, DeepFace, and ElevenLabs documentation.

3.3 User Classes and Characteristics

This section identifies different user categories interacting with the system. Each user class is defined along with their responsibilities, access privileges, and technical proficiency level.

Table 3.1: User Classes and Characteristics

User Class	Description	Technical Level
Administrator	Manages system configuration, user accounts, monitoring, and reports.	Advanced
End User (Candidate)	Uses the system to select job roles, conduct AI mock interviews, receive performance reports, and track improvement.	Basic
External System	AI APIs and third-party services (Whisper, Gemini, DeepFace, ElevenLabs) interacting with the application backend.	Technical

3.4 System Overview

The proposed system operates within a three-tier web architecture designed to support modular interaction between user interfaces, backend processing logic, AI services, and data storage. The system supports role-based interactions and ensures secure communication between internal modules and external AI services.

The system boundary includes:

- React.js-based frontend (user interface and webcam/microphone interaction).
- Node.js backend (business logic, API orchestration, session management).
- AI processing layer (speech-to-text, emotion detection, language model, voice synthesis).
- Database (user data, interview sessions, reports).

3.5 Use Case Model

The use case model represents high-level system interactions between actors and system functionalities. It defines system boundaries and captures functional behavior from a user perspective.

3.5.1 Use Case Diagram

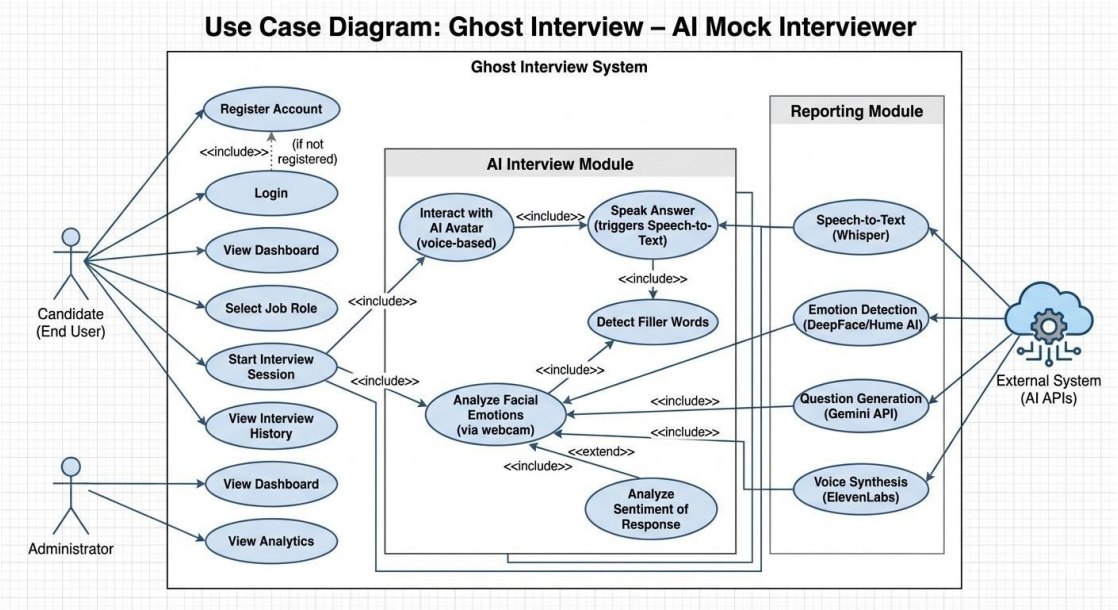


Figure 3.1: Use Case Diagram of Ghost Interview System

The diagram illustrates the interaction between the Candidate, Administrator, and external AI systems with the major system functionalities including login, job role selection, interview session, speech analysis, emotion analysis, and report generation.

3.5.2 Use Case Descriptions

3.5.2.1 Module 1: User Management

Table 3.2: UC-1.1: User Registration

Use Case ID	UC-1.1
Use Case Name	User Registration
Actors	End User (Candidate)
Description	This use case describes how a new user registers into the Ghost Interview system.
Trigger	User selects the Sign Up option on the homepage.
Preconditions	User is not already registered.
Postconditions	User account is successfully created and stored in the database.
Normal Flow	1. User navigates to the registration page. 2. User enters name, email, and password. 3. System validates input fields. 4. System stores data securely in the database. 5. Confirmation email is sent and success message is displayed.
Alternative Flow	If validation fails, the system displays appropriate error messages and prompts the user to correct the input.

Table 3.3: UC-1.2: User Login

Use Case ID	UC-1.2
Use Case Name	User Login
Actors	End User, Administrator
Description	This use case describes how a registered user logs into the system.
Trigger	User selects the Login option.
Preconditions	User has a registered account.
Postconditions	User is authenticated and redirected to the dashboard.
Normal Flow	1. User enters email and password. 2. System validates credentials against stored records. 3. System grants access and redirects to the dashboard.
Alternative Flow	If credentials are invalid, the system displays an error message and prompts re-entry.

3.5.2.2 Module 2: AI Interview Module

Table 3.4: UC-2.1: Job Role Selection and Interview Initiation

Use Case ID	UC-2.1
Use Case Name	Job Role Selection and Interview Initiation
Actors	End User
Description	The user selects a job role and initiates an AI-driven mock interview session.
Trigger	User clicks Start Interview after selecting a job role.
Preconditions	User is logged in; webcam and microphone are accessible.
Postconditions	Interview session is started and AI avatar begins asking questions.
Normal Flow	1. User selects a job role from the available list. 2. System loads relevant interview questions via Gemini API. 3. AI avatar voice is activated through ElevenLabs. 4. Avatar asks the first interview question. 5. Webcam and microphone recording begins.
Alternative Flow	If webcam or microphone access is denied, the system displays a permission error and halts the session.

Table 3.5: UC-2.2: Speech Analysis

Use Case ID	UC-2.2
Use Case Name	Speech Analysis
Actors	End User, External System (Whisper API)
Description	The system converts the user's spoken response to text and identifies filler words.
Trigger	User speaks in response to the AI avatar's question.
Preconditions	Interview session is active; microphone access is granted.
Postconditions	Text transcript and filler word count are stored for the session.
Normal Flow	1. User speaks their answer. 2. Audio is captured and sent to OpenAI Whisper API. 3. Whisper converts audio to text. 4. System scans transcript for filler words (e.g., "um", "uh"). 5. Results are stored in the session data.
Alternative Flow	If audio quality is too low, system flags low-quality recording and requests user to repeat.

Table 3.6: UC-2.3: Emotion and Sentiment Analysis

Use Case ID	UC-2.3
Use Case Name	Emotion and Sentiment Analysis
Actors	End User, External System (DeepFace / Hume AI)
Description	The system analyzes the user's facial expressions and emotional tone during the interview.
Trigger	Video frames are captured during the interview session.
Preconditions	Webcam access is granted; interview session is active.
Postconditions	Emotion metrics (confidence, nervousness, eye contact) are stored.
Normal Flow	1. Webcam frames are captured at regular intervals. 2. Frames are sent to DeepFace / Hume AI for analysis. 3. Emotion labels and confidence scores are returned. 4. Sentiment of text transcript is analyzed. 5. All metrics are stored in the session database.
Alternative Flow	If camera feed is unavailable, emotion analysis is skipped and user is notified.

3.5.2.3 Module 3: Reporting and Dashboard

Table 3.7: UC-3.1: Confidence and Competency Report Generation

Use Case ID	UC-3.1
Use Case Name	Confidence and Competency Report Generation
Actors	End User
Description	After the interview session, the system generates a detailed performance report covering speech, emotion, and competency metrics.
Trigger	Interview session is completed.
Preconditions	At least one interview session has been completed successfully.
Postconditions	Report is generated, stored, and displayed to the user.
Normal Flow	1. System aggregates speech, emotion, and sentiment data. 2. System computes confidence score, filler word frequency, and emotion timeline. 3. Report is formatted and stored in the database. 4. User is redirected to the Report screen with insights.
Alternative Flow	If session data is incomplete, system generates a partial report and notifies the user.

3.5.3 Event-Response Table

Table 3.8: Event-Response Table

Event	Source	System Response
User Login Attempt	End User	Validate credentials and grant or deny access.
Job Role Selection	End User	Load interview questions from Gemini API.
Interview Started	End User	Activate AI avatar, begin webcam and microphone recording.
User Speaks Answer	End User	Capture audio and send to Whisper for transcription.
Video Frame Captured	System	Send frame to emotion detection model; store result.
Interview Completed	System	Aggregate data and generate Confidence & Competency Report.
Report Requested	End User	Retrieve stored report and display on dashboard.
API Unavailability	External System	Display error message and pause session gracefully.

3.6 Functional Requirements

Functional requirements define the core services and behaviors that the system shall provide. These requirements are derived directly from the use case model. Each requirement is uniquely identified, measurable, and written using formal specification language following IEEE-style structure, categorized by system modules.

3.6.1 Module 1: User Management

3.6.1.1 FR-1.1 User Registration

Table 3.9: Functional Requirement – User Registration

Identifier	FR-1.1
Title	User Registration
Requirement	The system shall allow users to create an account by providing a valid name, email address, and password.
Source	Stakeholder Requirement
Rationale	Enables authorized access to system services and personalized interview tracking.
Dependencies	Database Connectivity
Priority	High

3.6.1.2 FR-1.2 User Authentication

Table 3.10: Functional Requirement – User Authentication

Identifier	FR-1.2
Title	User Authentication
Requirement	The system shall authenticate users using email and password and maintain secure sessions.
Source	Stakeholder Requirement
Rationale	Ensures only registered users can access interview and reporting features.
Dependencies	FR-1.1 (User Registration), Database Connectivity
Priority	High

3.6.2 Module 2: AI Interview Module

3.6.2.1 FR-2.1 Job Role Selection

Table 3.11: Functional Requirement – Job Role Selection

Identifier	FR-2.1
Title	Job Role Selection
Requirement	The system shall allow users to select from predefined job roles and interview categories before initiating a session.
Source	System Requirement Specification
Rationale	Enables domain-specific question generation tailored to the user's target role.
Dependencies	FR-1.2, Gemini API Integration
Priority	High

3.6.2.2 FR-2.2 AI Avatar Interview Conduction

Table 3.12: Functional Requirement – AI Avatar Interview

Identifier	FR-2.2
Title	AI Avatar Interview Conduction
Requirement	The system shall conduct voice-based interviews through an AI avatar that asks contextually relevant questions using a realistic human-like voice via the ElevenLabs API.
Source	System Requirement Specification
Rationale	Provides a realistic and immersive mock interview experience.
Dependencies	FR-2.1, Gemini API, ElevenLabs API, WebRTC
Priority	High

3.6.2.3 FR-2.3 Speech-to-Text Conversion

Table 3.13: Functional Requirement – Speech-to-Text Conversion

Identifier	FR-2.3
Title	Speech-to-Text Conversion
Requirement	The system shall convert the user's spoken responses into text using OpenAI Whisper and store the transcript for analysis.
Source	System Requirement Specification
Rationale	Enables verbal response evaluation and filler word detection.
Dependencies	OpenAI Whisper API, Microphone Access via WebRTC
Priority	High

3.6.2.4 FR-2.4 Filler Word Detection

Table 3.14: Functional Requirement – Filler Word Detection

Identifier	FR-2.4
Title	Filler Word Detection
Requirement	The system shall identify and count filler words (e.g., "um", "uh", "like", "you know") in the speech transcript.
Source	System Requirement Specification
Rationale	Provides actionable feedback on speech quality and fluency.
Dependencies	FR-2.3 (Speech-to-Text)
Priority	Medium

3.6.2.5 FR-2.5 Facial Emotion Detection

Table 3.15: Functional Requirement – Facial Emotion Detection

Identifier	FR-2.5
Title	Facial Emotion Detection
Requirement	The system shall analyze webcam video frames using DeepFace or Hume AI to detect facial emotions, eye contact, and confidence indicators during the interview.
Source	System Requirement Specification
Rationale	Provides non-verbal behavior analysis, a key component of comprehensive interview assessment.
Dependencies	Webcam Access via WebRTC, DeepFace / Hume AI API
Priority	High

3.6.2.6 FR-2.6 Sentiment Analysis

Table 3.16: Functional Requirement – Sentiment Analysis

Identifier	FR-2.6
Title	Sentiment Analysis
Requirement	The system shall evaluate the emotional tone (positive, negative, neutral) of the user's transcribed responses using NLP-based sentiment analysis.
Source	System Requirement Specification
Rationale	Complements emotion detection with language-level emotional assessment.
Dependencies	FR-2.3 (Speech-to-Text)
Priority	Medium

3.6.3 Module 3: Reporting and Analytics

3.6.3.1 FR-3.1 Confidence and Competency Report Generation

Table 3.17: Functional Requirement – Report Generation

Identifier	FR-3.1
Title	Confidence and Competency Report Generation
Requirement	The system shall generate a detailed Confidence and Competency Report after each interview session, including speech quality, emotion timeline, filler word count, sentiment summary, and an overall readiness score.
Source	Business Requirement
Rationale	Supports user self-assessment and performance improvement.
Dependencies	FR-2.3, FR-2.4, FR-2.5, FR-2.6, Database Module
Priority	High

3.6.3.2 FR-3.2 Interview History Dashboard

Table 3.18: Functional Requirement – Interview History Dashboard

Identifier	FR-3.2
Title	Interview History Dashboard
Requirement	The system shall provide users with a dashboard displaying their interview history, past reports, performance trends, and analytics.
Source	Business Requirement
Rationale	Enables users to track improvement over time and review past sessions.
Dependencies	FR-3.1, Database Module, FR-1.2
Priority	Medium

3.7 Non-Functional Requirements

Non-functional requirements define quality attributes that describe how the system performs. These requirements are measurable and support system reliability, usability, performance, security, and maintainability.

3.7.1 Performance Requirements

- **PER-1:** The system shall load the main dashboard within 3 seconds under normal operating conditions.
- **PER-2:** Speech-to-text transcription responses from OpenAI Whisper shall be returned within 5 seconds of audio submission.
- **PER-3:** Emotion detection results shall be processed within 2 seconds per video frame batch.
- **PER-4:** The system shall handle at least 50 concurrent interview sessions without significant degradation in response time.
- **PER-5:** The AI avatar's first question shall be delivered within 3 seconds of the interview session start.

3.7.2 Security Requirements

- **SEC-1:** The system shall implement secure user authentication using hashed passwords (e.g., bcrypt).
- **SEC-2:** All data transmission shall use HTTPS protocol to ensure encryption in transit.
- **SEC-3:** Webcam and microphone access shall require explicit user permission via browser consent prompts.
- **SEC-4:** Role-based access control shall be enforced (Admin vs Candidate).
- **SEC-5:** API keys for third-party services shall be stored server-side and never exposed to the client.
- **SEC-6:** User interview recordings and session data shall be stored securely and not shared with third parties.

3.7.3 Reliability Requirements

- **REL-1:** The system shall maintain at least 99% uptime during the evaluation period.
- **REL-2:** System failures shall not result in loss of completed interview session data.
- **REL-3:** The system shall implement error handling for third-party API unavailability and notify users gracefully.
- **REL-4:** Backup and recovery mechanisms shall be implemented for the database.

3.7.4 Usability Requirements

- **USE-1:** The interface shall be intuitive and easy to navigate without prior training.
- **USE-2:** Clear and meaningful error messages shall be displayed for all failure scenarios.
- **USE-3:** The system shall follow standard UI/UX design principles and be visually appealing.

- **USE-4:** The application shall be fully responsive across desktop and laptop screen sizes.
- **USE-5:** The AI avatar interaction interface shall clearly indicate when the avatar is speaking and when the user is expected to respond.

3.7.5 Scalability Requirements

- **SCA-1:** The system architecture shall support increasing number of registered users and concurrent sessions.
- **SCA-2:** The backend shall allow horizontal scaling through cloud deployment.
- **SCA-3:** The database design shall support large volumes of session transcripts and report data.
- **SCA-4:** New job roles and interview categories shall be addable without major restructuring.

3.7.6 Maintainability Requirements

- **MAIN-1:** The system shall follow modular architecture principles with clearly separated frontend, backend, and AI service layers.
- **MAIN-2:** Source code shall be documented clearly using inline comments and README files.
- **MAIN-3:** Future updates, including additional AI models or job roles, shall be integrable without major system restructuring.
- **MAIN-4:** The system shall use version-controlled repositories (e.g., GitHub) for code management.

3.8 External Interface Requirements

This section defines how the system interacts with external entities including users, hardware devices, software systems, and communication networks.

3.8.1 User Interface Requirements

- **UI-1:** The system shall provide a graphical web-based user interface accessible via modern browsers.
- **UI-2:** Standard font sizes (12–14 pt) shall ensure readability throughout the application.
- **UI-3:** Layout shall be responsive across desktop and large-screen devices.
- **UI-4:** Navigation shall remain consistent across all modules (login, role selection, interview, report, dashboard).
- **UI-5:** The interview screen shall display a live webcam feed, AI avatar, and question text simultaneously.

3.8.2 Hardware Interface Requirements

- **HI-1:** The system shall operate on standard computing devices (PC, laptop) with a modern web browser.
- **HI-2:** The system requires access to a functional webcam and microphone for interview session functionality.

- **HI-3:** Minimum hardware: 4GB RAM, modern dual-core processor, HD webcam (720p or higher recommended for accurate emotion detection).

3.8.3 Software Interface Requirements

- **SI-1:** The system shall interact with a relational or non-relational database for storing user data, sessions, and reports.
- **SI-2:** The system shall integrate with the Gemini API for AI-driven interview question generation.
- **SI-3:** The system shall integrate with OpenAI Whisper API for speech-to-text conversion.
- **SI-4:** The system shall integrate with DeepFace / Hume AI for facial emotion detection.
- **SI-5:** The system shall integrate with ElevenLabs API for realistic AI voice synthesis.
- **SI-6:** The frontend shall be built using React.js (v18+) and the backend using Node.js (v20+).

3.8.4 Communication Interface Requirements

- **CI-1:** The system shall use HTTPS protocol for all secure client-server communication.
- **CI-2:** RESTful APIs shall be used for backend communication between frontend and server.
- **CI-3:** WebRTC shall be used for real-time webcam and microphone access within the browser.
- **CI-4:** All third-party API calls shall be made server-side to protect API credentials.

3.9 Assumptions and Dependencies

- The system assumes stable internet connectivity for all users during interview sessions.
- Third-party services (Gemini, Whisper, DeepFace, ElevenLabs) are assumed to remain available and functional throughout the project.
- Users are assumed to possess basic digital literacy and be comfortable using web browsers.
- Required cloud hosting infrastructure is assumed to be available during deployment.
- Users are assumed to have access to a functional HD webcam and microphone.
- Pre-trained AI models will be used; no custom model training is assumed within project scope.

3.10 Requirement Traceability Matrix

This matrix maps project objectives to functional requirements to ensure alignment and completeness.

Table 3.19: Requirement Traceability Matrix

Objective	Use Case	Functional Requirement
Provide user access management	UC-1.1, UC-1.2	FR-1.1, FR-1.2
Conduct AI mock interview sessions	UC-2.1, UC-2.2	FR-2.1, FR-2.2, FR-2.3
Analyze speech quality	UC-2.2	FR-2.3, FR-2.4
Analyze non-verbal behavior	UC-2.3	FR-2.5, FR-2.6
Generate performance reports	UC-3.1	FR-3.1
Provide interview history & analytics	UC-3.1	FR-3.2

3.11 Chapter Summary

This chapter presented a structured requirement analysis of the Ghost Interview AI Mock Interviewer system. It defined requirement elicitation techniques, user classes, use case models, functional requirements, non-functional requirements, and external interface requirements. A traceability matrix was included to ensure alignment between objectives and implementation requirements. These requirements form the formal foundation for the system design presented in the next chapter.

Chapter 4

Software Design

4.1 Introduction to Software Design

This chapter presents the detailed software design of the Ghost Interview AI Mock Interviewer system. It translates the requirements identified in Chapter 3 into structured architectural, behavioral, and data models. The purpose of this chapter is to provide a clear blueprint for system implementation, ensuring maintainability, scalability, and modular development.

This chapter includes architectural design, UML models, data flow models, and database design for the Ghost Interview web-based AI application. The system integrates multiple AI services including OpenAI Whisper, Gemini API, DeepFace/Hume AI, and ElevenLabs to deliver a comprehensive mock interview experience.

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodology

This project follows the Object-Oriented Design (OOD) methodology combined with a Layered Architecture approach. This methodology is selected due to the following reasons:

- **Modularity:** Each system module (User Management, AI Interview, Speech Analysis, Emotion Detection, Reporting) is developed as an independent, reusable component.
- **Scalability:** The layered architecture allows each tier (frontend, backend, AI services) to be scaled independently based on load.
- **Loose Coupling:** AI service integrations (Whisper, Gemini, ElevenLabs, DeepFace) are loosely coupled through API abstraction layers, allowing easy replacement or upgrade of individual services.
- **Reusability:** Common components such as authentication, session management, and report generation are reused across modules.

The design follows key principles including SOLID, DRY (Don’t Repeat Yourself), and Separation of Concerns, ensuring clean and maintainable code architecture.

4.2.2 Software Process Model

The project follows the Agile Incremental development model. This model is selected for the following reasons:

- The system integrates multiple AI APIs whose behavior may require iterative refinement based on real testing results.
- Features such as emotion detection accuracy and speech analysis quality need continuous evaluation and improvement cycles.
- Agile allows the team to deliver working modules incrementally: first user management, then the interview engine, then analysis modules, and finally the reporting dashboard.
- Testing and validation are integrated at the end of each sprint, reducing late-stage defect discovery.

Development Phases / Sprints

1. **Sprint 1:** User registration, login, and dashboard setup.
1. **Sprint 2:** Job role selection and Gemini API interview question integration.
2. **Sprint 3:** AI avatar voice interaction using ElevenLabs and WebRTC.
3. **Sprint 4:** Speech-to-text conversion (Whisper) and filler word detection.
4. **Sprint 5:** Emotion detection (DeepFace/Hume AI) and sentiment analysis.
5. **Sprint 6:** Confidence & Competency Report generation and interview history dashboard.
6. **Sprint 7:** Testing, optimization, and final deployment.

4.3 System Overview

This section provides a high-level overview of Ghost Interview system components and their interactions with external entities.

External Actors

- **Candidate (End User):** Interacts with the system through a web browser using webcam and microphone.
- **Administrator:** Manages system configuration and monitors users.
- **Gemini API:** Provides AI-generated interview questions based on job role.
- **OpenAI Whisper API:** Converts candidate's speech to text.
- **DeepFace / Hume AI:** Analyzes facial emotions from webcam frames.
- **ElevenLabs API:** Synthesizes realistic AI avatar voice for questions.
- **Database:** Stores user data, session transcripts, and generated reports.

Major Subsystems

- User Management Subsystem
- AI Interview Engine Subsystem
- Speech & Sentiment Analysis Subsystem
- Emotion Detection Subsystem
- Reporting & Analytics Subsystem

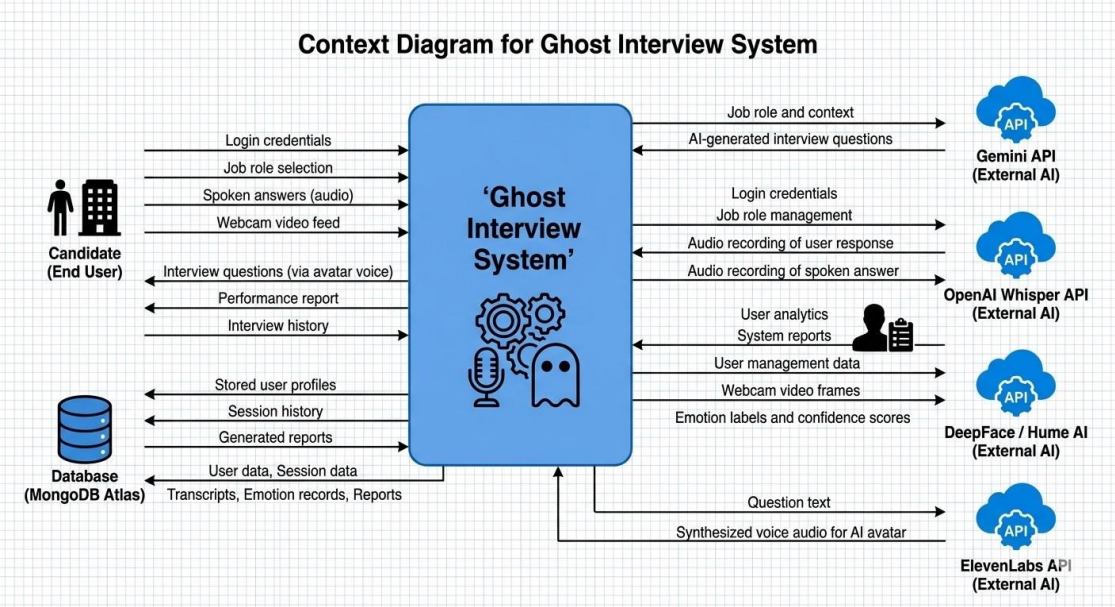


Figure 4.1: Context Diagram of Ghost Interview System

4.4 Architectural Design

This section describes the high-level three-tier architecture of the Ghost Interview system. The system is divided into the following layers:

- **Presentation Layer (Frontend):** Built using React.js (v18+). Handles all user interface components including login, job role selection, interview screen (webcam + avatar), and report dashboard. WebRTC is used for real-time webcam and microphone access.
- **Business Logic Layer (Backend):** Built using Node.js (v20+). Handles session management, API orchestration, speech analysis pipeline, emotion data aggregation, and report computation. RESTful APIs are exposed to the frontend.
- **AI Services Layer:** Comprises four external AI APIs:
  - Gemini API – Interview question generation.
  - OpenAI Whisper – Speech-to-text conversion.
  - DeepFace / Hume AI – Facial emotion detection.
  - ElevenLabs – AI voice synthesis for avatar.
- **Data Layer (Database):** Stores all persistent data including user accounts, interview sessions, transcripts, emotion metrics, and generated reports. A NoSQL database (e.g., MongoDB) is preferred for its flexibility with session and report data structures.

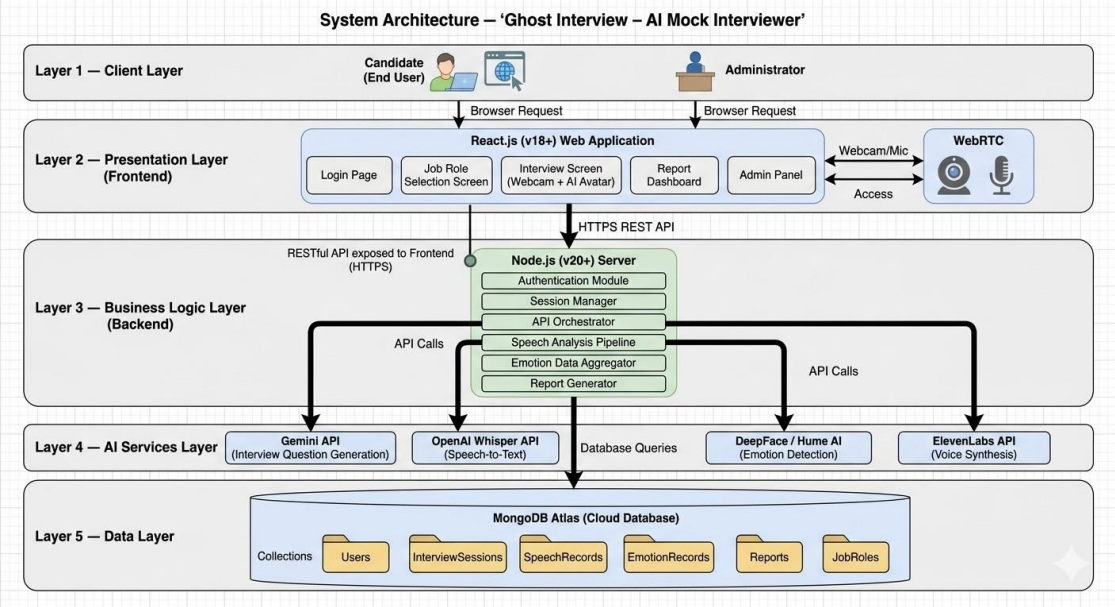


Figure 4.2: System Architecture Diagram of Ghost Interview

Component Interaction: The frontend communicates with the backend via RESTful APIs over HTTPS. The backend orchestrates calls to the four AI service APIs, processes responses, and stores results in the database. All API keys are stored server-side for security. WebRTC streams webcam and audio data directly in the browser, which is then captured and forwarded to the backend for processing.

4.5 Design Models

This section presents detailed UML design models of the Ghost Interview system. These models describe system behavior, structure, and interactions, helping developers understand system workflow before implementation.

4.5.1 Activity Diagrams

Activity diagrams represent the workflow of the different modules of the system.

4.5.1.1 Module 1: User Management

Description: This diagram illustrates the user registration, login, authentication, and profile management processes for Ghost Interview.

1. User navigates to the registration or login page.
1. For registration: User enters name, email, and password → System validates input → Account created → Confirmation displayed.
2. For login: User enters credentials → System validates against database → If valid, session created and user redirected to dashboard → If invalid, error message displayed.
3. User can update profile information from the dashboard.

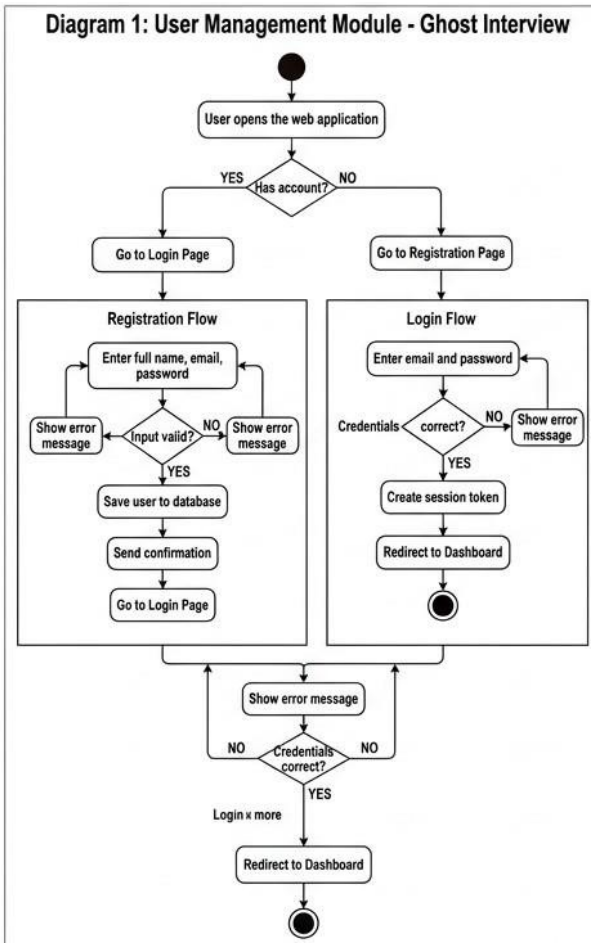


Figure 4.3: Activity Diagram – User Management Module

4.5.1.2 Module 2: AI Interview Core Module

Description: This diagram illustrates the core interview session workflow including job role selection, avatar interaction, speech capture, and analysis.

1. User selects a job role from predefined categories.
1. System requests interview questions from Gemini API.
2. Webcam and microphone permissions are requested via WebRTC.
3. AI avatar delivers the first question using ElevenLabs voice.
4. User speaks their answer → Audio captured → Sent to Whisper for transcription.
5. Webcam frames captured simultaneously → Sent to DeepFace/Hume AI for emotion analysis.
6. System checks if all questions are answered → If yes, session ends and report is generated → If no, next question is delivered.

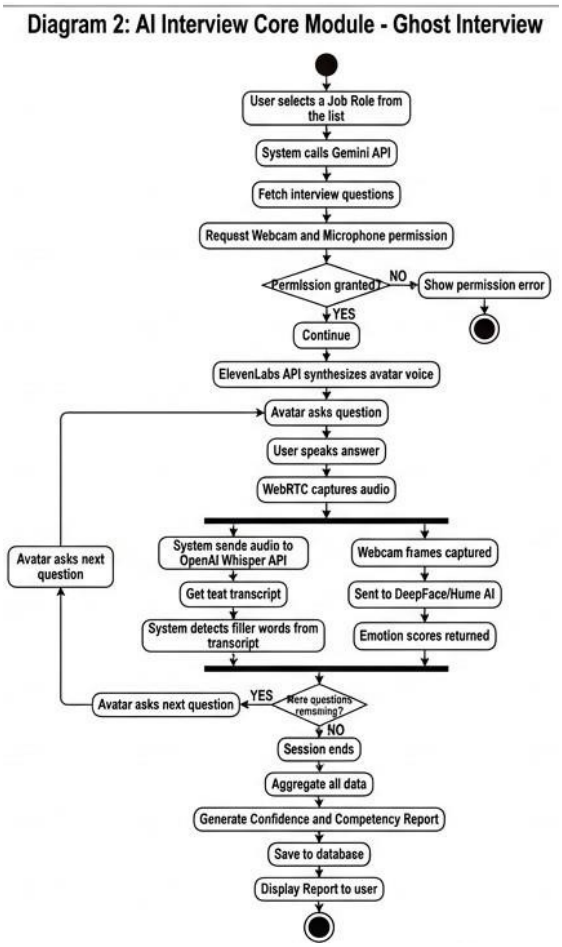


Figure 4.4: Activity Diagram – AI Interview Core Module

4.5.1.3 Module 3: Admin Module

Description: This diagram illustrates the administrator's operations within the Ghost Interview system.

1. Admin logs in with elevated credentials.
1. Admin accesses the admin panel.
2. Admin can monitor registered users and their interview history.
3. Admin can manage job roles and interview question categories.
4. Admin can generate system-wide reports and analytics.
5. Admin can deactivate or remove user accounts.
6. Admin logs out.

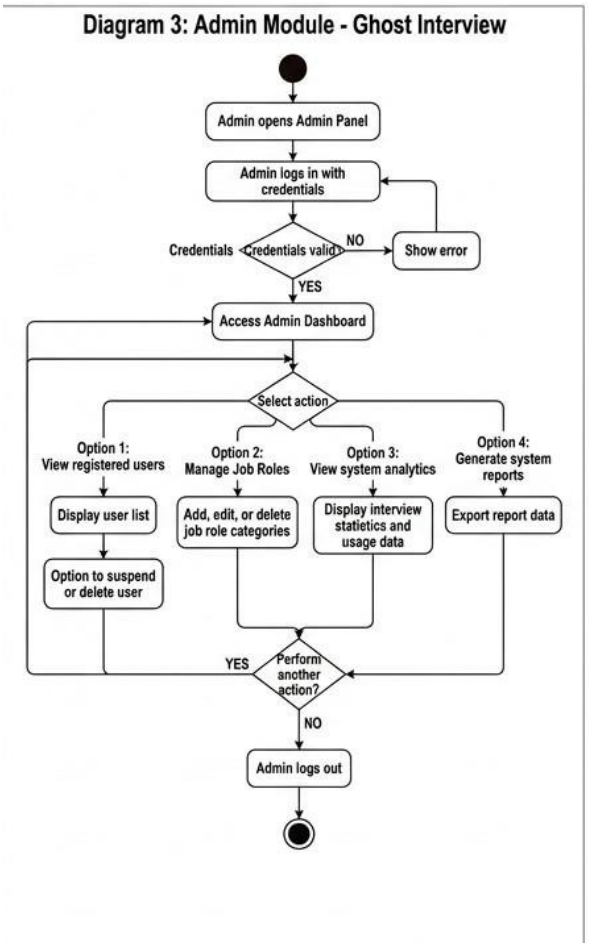


Figure 4.5: Activity Diagram – Admin Module

4.5.2 Class Diagram

The class diagram shows the static structure of the Ghost Interview system, including all major classes, their attributes, methods, and relationships.

Key Classes

- **User:** Stores user profile and authentication data. Methods include register(), login(), logout(), updateProfile().
- **InterviewSession:** Manages a single interview session. Attributes include session ID, job role, start/end time, and status. Methods include startSession(), endSession(), getQuestions().
- **SpeechAnalyzer:** Handles audio processing. Methods include captureAudio(), transcribeSpeech(), detectFillerWords().
- **EmotionAnalyzer:** Handles webcam frame analysis. Methods include captureFrame(), analyzeEmotion(), getSentiment().
- **Report:** Generates and stores performance reports. Methods include generateReport(), calculateConfidenceScore(), displayReport().
- **Admin:** Extends User with elevated privileges. Methods include manageUsers(), manageJobRoles(), viewSystemAnalytics().
- **JobRole:** Stores predefined job role categories and associated question sets.

Relationships

- User has many InterviewSessions (1:N).
- InterviewSession uses SpeechAnalyzer and EmotionAnalyzer (Association).
- InterviewSession generates one Report (1:1).
- Admin inherits from User (Inheritance).
- InterviewSession is associated with one JobRole (Association).

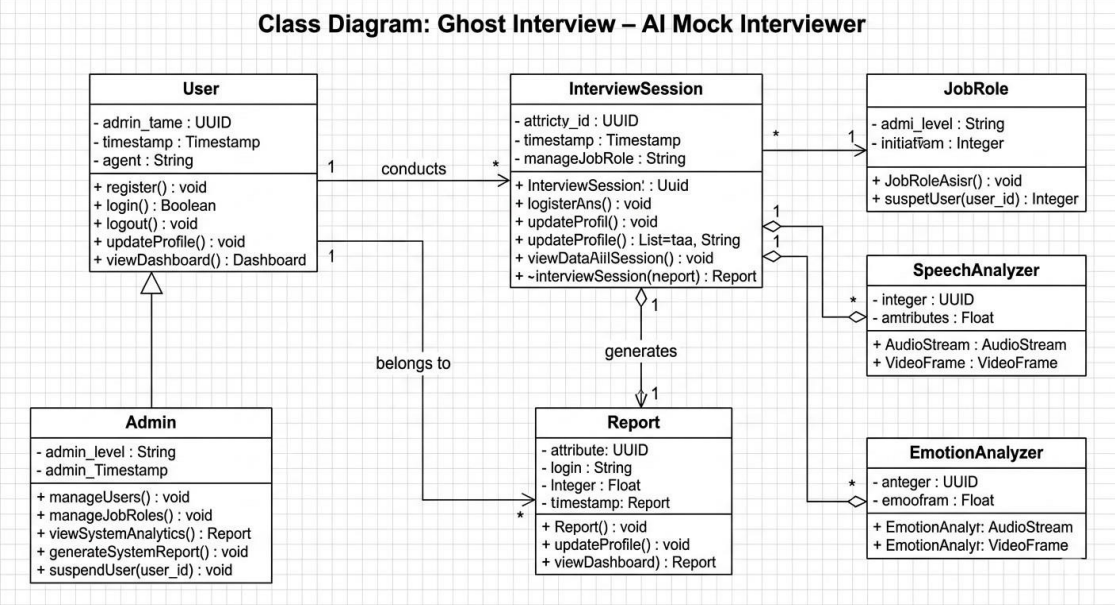


Figure 4.6: Class Diagram of Ghost Interview System

4.5.3 Sequence Diagrams

Sequence diagrams show the interaction between objects over time for major system operations.

4.5.3.1 Sequence Diagram – User Login

Description: This sequence diagram shows the login flow for a registered user.

1. User enters email and password on the login page.
1. Frontend sends a POST request to the Backend (/api/login).
2. Backend queries the Database for user credentials.
3. Database returns user record.
4. Backend validates password hash.
5. If valid: Backend creates a session token and returns success response to Frontend.
6. Frontend redirects user to the dashboard.
7. If invalid: Backend returns an error response and Frontend displays error message.

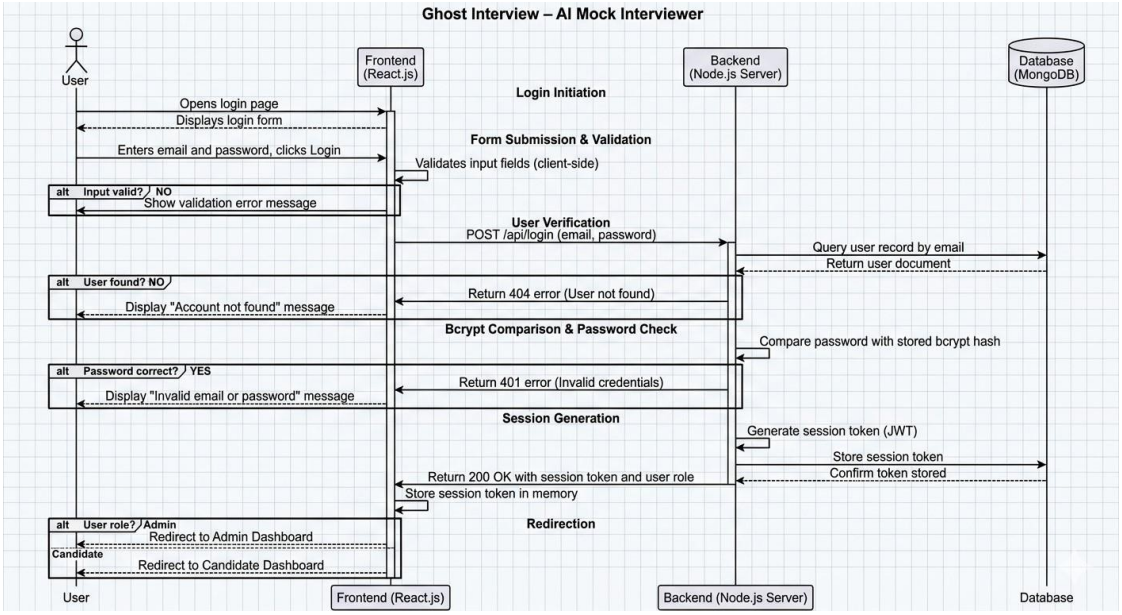


Figure 4.7: Sequence Diagram – User Login

4.5.3.2 Sequence Diagram – AI Interview Core Operation

Description: This sequence diagram shows the complete interview session flow from job role selection to report generation.

1. User selects a job role → Frontend sends request to Backend.
1. Backend calls Gemini API → Gemini returns interview questions.
2. Backend sends first question text to Frontend.
3. Frontend calls ElevenLabs API → ElevenLabs returns audio stream → AI avatar speaks the question.
4. User speaks answer → WebRTC captures audio.
5. Frontend sends audio to Backend → Backend calls Whisper API → Whisper returns text transcript.
6. Backend detects filler words from transcript and stores result.
7. Simultaneously, Frontend captures webcam frames → Backend calls DeepFace/Hume AI → Emotion scores returned and stored.
8. Process repeats for all questions.
9. On session end: Backend aggregates all data → Generates Confidence & Competency Report → Stores in Database → Returns report to Frontend.
10. Frontend displays the report to the user.

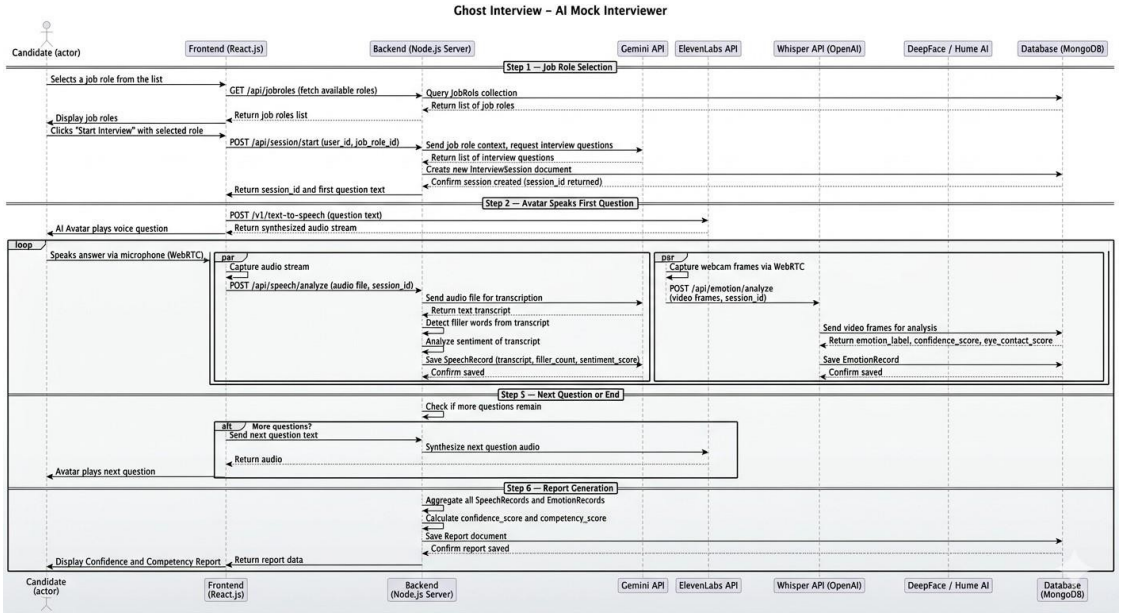


Figure 4.8: Sequence Diagram – AI Interview Core Operation

4.5.4 State Transition Diagrams

State diagrams show how the system or an object changes states over time based on events and conditions.

4.5.4.1 Interview Session States

- Idle → (User selects job role) → Initializing
- Initializing → (APIs loaded, webcam granted) → Active
- Active → (All questions answered) → Processing
- Processing → (Report generated) → Completed
- Active → (User quits early) → Terminated
- Active → (API failure) → Error
- Error → (User retries) → Initializing

4.5.4.2 User Account States

- Registered → Active → Suspended → Deleted

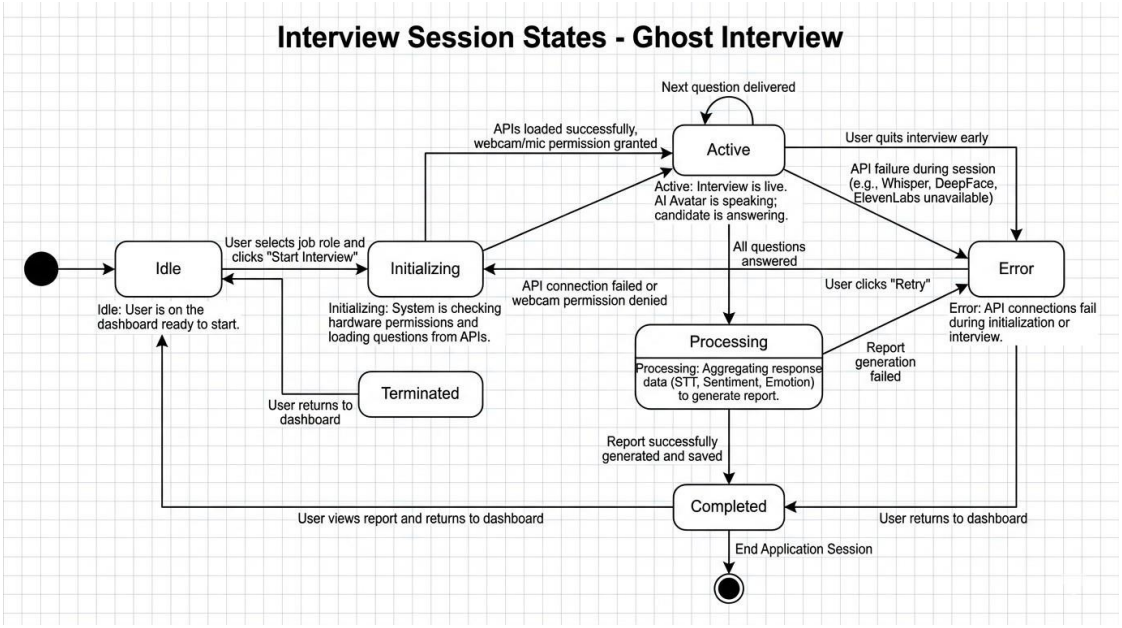


Figure 4.9: State Transition Diagram – Interview Session

4.6 Data Flow Diagrams

Data Flow Diagrams (DFD) describe how data moves within the Ghost Interview system. Different levels of DFD provide increasing detail.

4.6.1 Level 1 Data Flow Diagram

Level 1 DFD decomposes the Ghost Interview system into its major sub-processes and shows how data flows between them.

Major Processes

- **P1 Authentication Module:** Handles user registration and login. Data flows: User credentials → P1 → User Database.
- **P2 Interview Engine Module:** Manages job role selection and avatar interaction. Data flows: Job role → P2 → Gemini API → Questions → ElevenLabs → Audio.
- **P3 Speech Analysis Module:** Processes audio responses. Data flows: Audio → P3 → Whisper API → Transcript → Session Store.
- **P4 Emotion Detection Module:** Processes webcam frames. Data flows: Frames → P4 → DeepFace/Hume AI → Emotion scores → Session Store.
- **P5 Reporting Module:** Generates performance reports. Data flows: Session Store → P5 → Report Database → User Dashboard.
- **P6 Admin Module:** System monitoring and management. Data flows: Admin → P6 → User Database / Report Database.

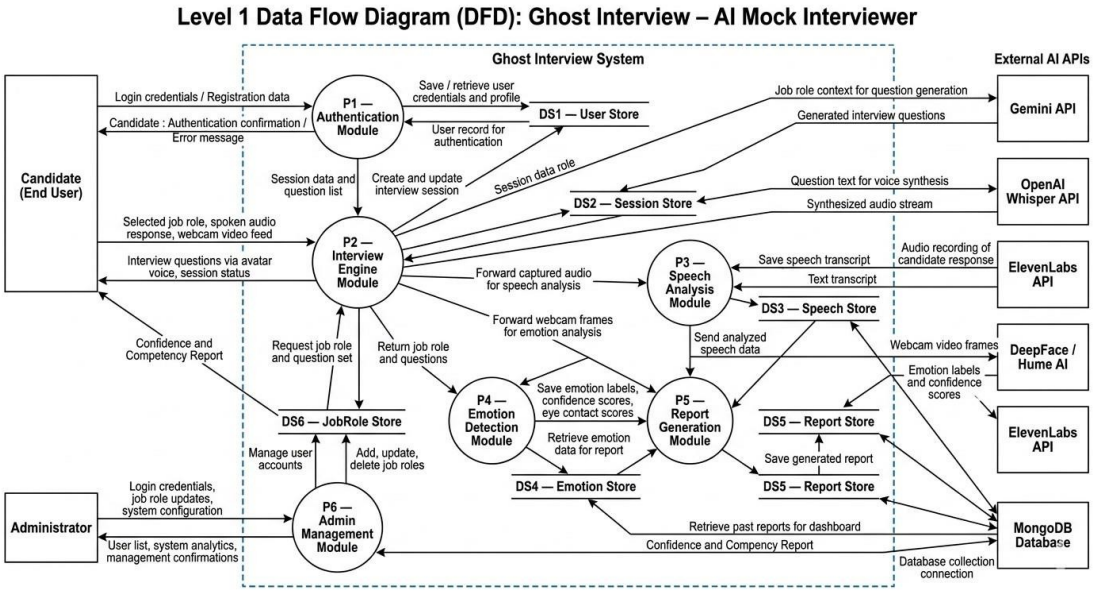


Figure 4.10: Level 1 Data Flow Diagram – Ghost Interview System

4.6.2 Level 2 Data Flow Diagram

Level 2 DFD further decomposes the AI Interview Engine Module (P2) from Level 1 into detailed sub-processes.

Sub-processes of P2 – Interview Engine

- **P2.1 Job Role Loader:** Fetches predefined job roles from the database and presents them to the user.
- **P2.2 Question Generator:** Sends job role data to Gemini API and retrieves a list of interview questions.
- **P2.3 Avatar Voice Synthesizer:** Sends question text to ElevenLabs API and plays back the generated audio through the frontend.
- **P2.4 Response Collector:** Captures user's spoken response via WebRTC microphone and forwards audio to the Speech Analysis Module (P3).
- **P2.5 Session Controller:** Tracks question index, manages timing, and triggers session end when all questions are completed.

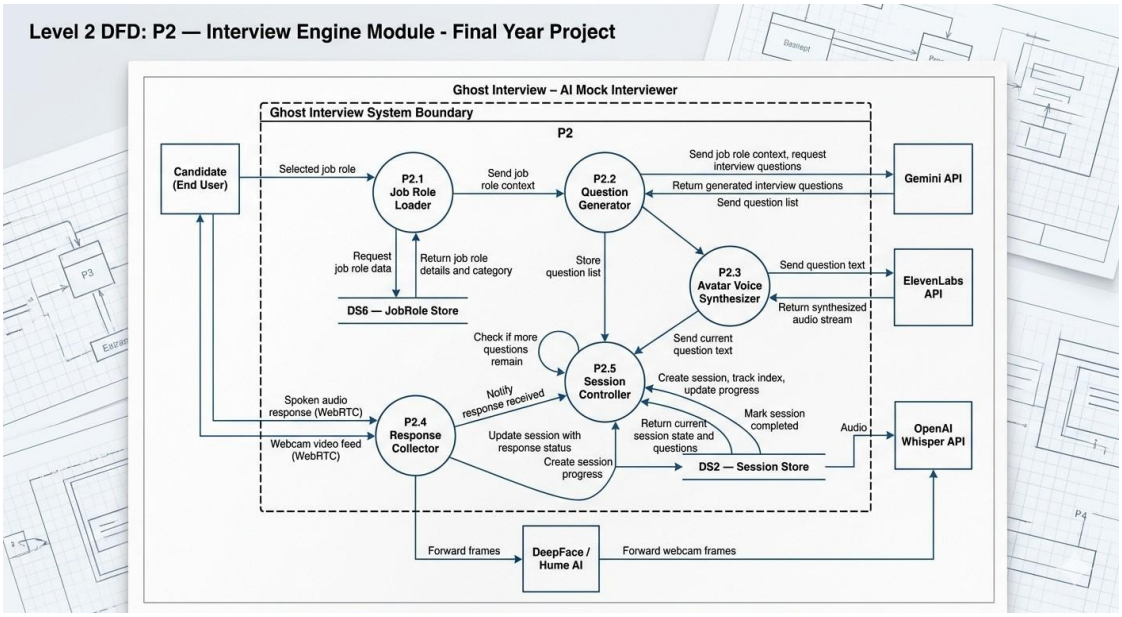


Figure 4.11: Level 2 Data Flow Diagram – AI Interview Engine Module

4.7 Data Design

This section describes how data is structured, stored, and managed within the Ghost Interview system.

Database Type: The system uses a NoSQL database (MongoDB) for the following reasons:

- Interview session data and emotion metrics have variable and nested structures that are better represented as JSON documents.
- MongoDB's flexible schema accommodates future additions to report structure without migrations.
- High read/write throughput supports real-time session data storage during active interviews.

Indexing Strategy: Indexes are applied on user_id, session_id, and created_at fields for efficient querying of user sessions and reports.

Security: Passwords are hashed using bcrypt. All API keys are stored as environment variables server-side. Sensitive session data is accessible only to the authenticated user.

Scalability: MongoDB supports horizontal scaling through sharding. Collections are designed to support large volumes of transcript and emotion metric documents.

4.7.1 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram represents the logical data structure of Ghost Interview.

Entities

- **User** (user_id PK, full_name, email, password_hash, role, created_at, status)
- **InterviewSession** (session_id PK, user_id FK, job_role_id FK, start_time, end_time, status)
- **JobRole** (job_role_id PK, role_name, category, description)
- **SpeechRecord** (speech_id PK, session_id FK, transcript, filler_word_count, sentiment_score, recorded_at)

- **EmotionRecord** (emotion_id PK, session_id FK, emotion_label, confidence_score, eye_contact_score, captured_at)
- **Report** (report_id PK, session_id FK, user_id FK, confidence_score, competency_score, filler_count, emotion_summary, generated_at)

Relationships

- User conducts many InterviewSessions (1:N).
- InterviewSession belongs to one JobRole (N:1).
- InterviewSession has one SpeechRecord per question (1:N).
- InterviewSession has many EmotionRecords (1:N).
- InterviewSession generates one Report (1:1).

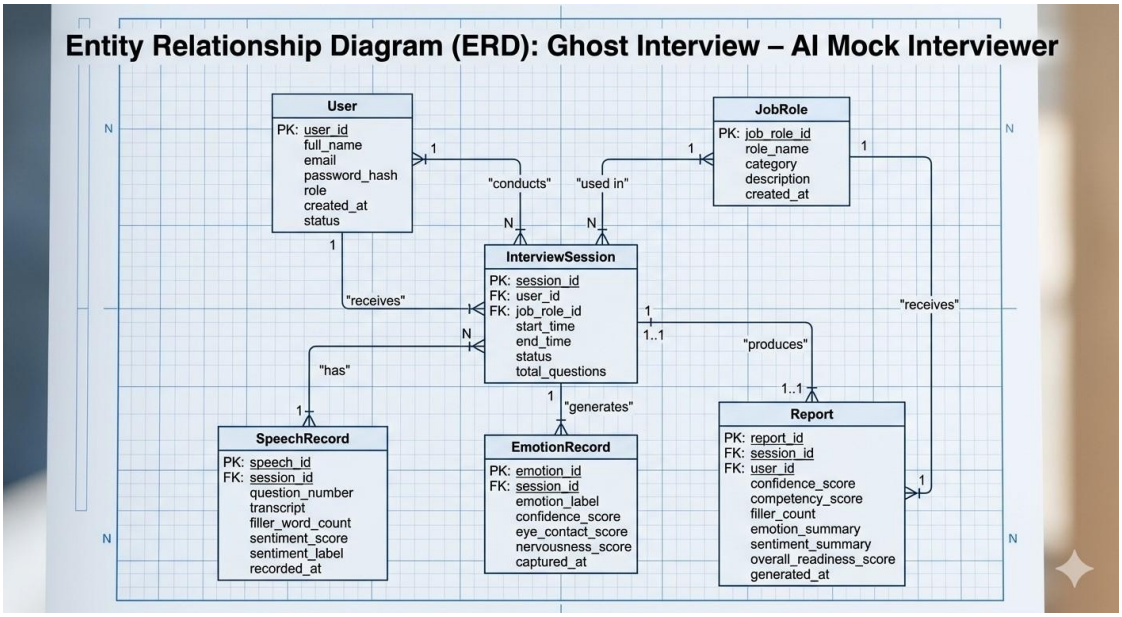


Figure 4.12: Entity Relationship Diagram – Ghost Interview System

4.7.2 Database Architecture

The Ghost Interview system uses a Client-Server Architecture with the following database design:

- **Client-Server Architecture:** The React.js frontend communicates with the Node.js backend server, which in turn queries the MongoDB database.
- **Cloud Storage:** The database is hosted on MongoDB Atlas (cloud-managed), ensuring high availability and automated backups.
- **Session Data:** Interview session documents, transcripts, and emotion records are stored as nested JSON documents within MongoDB collections.

4.8 Database Schema Diagram

The Database Schema Diagram represents the physical database implementation structure of the Ghost Interview system.

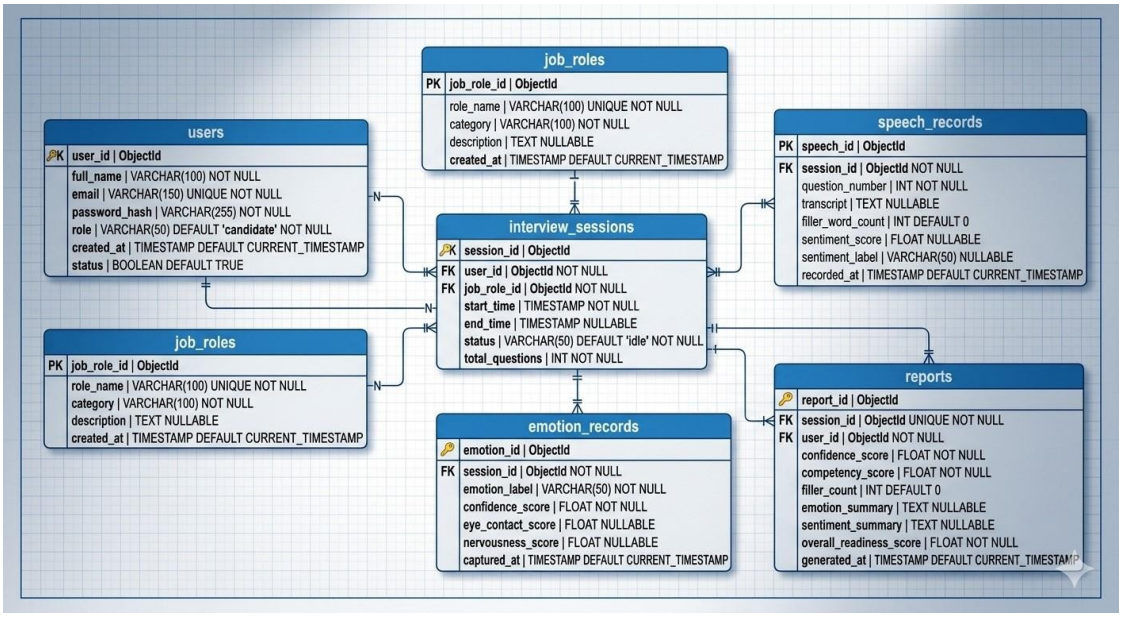


Figure 4.13: Database Schema Diagram – Ghost Interview System

The schema is designed to support:

- **Data Integrity:** Foreign key-equivalent references between collections ensure consistent relationships between users, sessions, and reports.
- **Efficient Querying:** Indexed fields on user_id and session_id reduce query response times.
- **Scalability:** Document-based storage allows horizontal scaling via MongoDB Atlas sharding.
- **Security:** Password fields store only bcrypt hashes; API keys are never stored in the database.

4.8.1 Data Dictionary

The Data Dictionary defines the structure of all major data elements in the system.

4.9 Database Tables / Collections

This section describes the logical database design of the Ghost Interview system. The database is structured to ensure data consistency, integrity, security, and scalability.

Table 4.1: Data Dictionary – User Collection

Field Name	Type	Size	Description
user_id	UUID / ObjectId	—	Unique identifier for each user (Primary Key)
full_name	VARCHAR	100	Full name of the registered user
email	VARCHAR	150	Unique registered email address used for login
password_hash	VARCHAR	255	Bcrypt-hashed password for secure authentication
role	VARCHAR	50	Access level: Admin or Candidate
created_at	TIMESTAMP	—	Account creation date and time
status	BOOLEAN	—	Account active (true) or suspended (false)

Table 4.2: Data Dictionary – InterviewSession Collection

Field Name	Type	Size	Description
session_id	UUID / ObjectId	—	Unique session identifier (Primary Key)
user_id	UUID / ObjectId	—	Reference to the User collection (Foreign Key)
job_role_id	UUID / ObjectId	—	Reference to the JobRole collection (Foreign Key)
start_time	TIMESTAMP	—	Timestamp when the interview session started
end_time	TIMESTAMP	—	Timestamp when the interview session ended
status	VARCHAR	50	Session state: Active, Completed, Terminated, Error

Table 4.3: Data Dictionary – SpeechRecord Collection

Field Name	Type	Size	Description
speech_id	UUID / ObjectId	—	Unique speech record identifier (Primary Key)
session_id	UUID / ObjectId	—	Reference to InterviewSession (Foreign Key)
transcript	TEXT	—	Full text transcript of the user’s spoken answer
filler_word_count	INT	—	Total number of filler words detected in the response
sentiment_score	FLOAT	—	Sentiment polarity score (positive/neutral/negative)
recorded_at	TIMESTAMP	—	Timestamp of the speech recording

Table 4.4: Data Dictionary – EmotionRecord Collection

Field Name	Type	Size	Description
emotion_id	UUID / ObjectId	—	Unique emotion record identifier (Primary Key)
session_id	UUID / ObjectId	—	Reference to InterviewSession (Foreign Key)
emotion_label	VARCHAR	50	Detected emotion: Happy, Neutral, Nervous, Angry, etc.
confidence_score	FLOAT	—	AI model confidence score for the detected emotion
eye_contact_score	FLOAT	—	Score indicating level of eye contact with camera
captured_at	TIMESTAMP	—	Timestamp when the webcam frame was captured

Table 4.5: Data Dictionary – Report Collection

Field Name	Type	Size	Description
report_id	UUID / ObjectId	—	Unique report identifier (Primary Key)
session_id	UUID / ObjectId	—	Reference to InterviewSession (Foreign Key)
user_id	UUID / ObjectId	—	Reference to the User collection (Foreign Key)
confidence_score	FLOAT	—	Overall confidence score computed from emotion data
competency_score	FLOAT	—	Overall competency score based on speech quality
filler_count	INT	—	Total filler words across the entire session
emotion_summary	TEXT	—	JSON summary of emotion distribution during session
generated_at	TIMESTAMP	—	Timestamp when the report was generated

4.9.1 User Collection

The User collection stores authentication credentials and user profile information. It supports role-based access control and secure authentication.

- **Primary Key:** user_id
- **Relationships:** Linked to InterviewSession and Report collections.
- **Authentication Attributes:** Email, password hash, role, account status.

Table 4.6: User Collection Structure

Field Name	Data Type	Constraint	Description
user_id	UUID / ObjectId	Primary Key	Unique identifier for each user
full_name	VARCHAR(100)	Not Null	User's full name
email	VARCHAR(150)	Unique, Not Null	Login email address
password_hash	VARCHAR(255)	Not Null	Bcrypt-encrypted password
role	VARCHAR(50)	Not Null	Admin or Candidate
created_at	TIMESTAMP	Default Current	Account creation timestamp
status	BOOLEAN	Default True	Active or suspended account

4.9.2 Core Data Collections

These collections store the primary operational data processed by the Ghost Interview system.

- **InterviewSession Collection:** Stores metadata for each mock interview session conducted by a user.
- **JobRole Collection:** Stores predefined job categories (e.g., Software Engineer, Data Analyst, HR Manager) and associated interview question sets.
- **SpeechRecord Collection:** Stores the text transcript, filler word count, and sentiment score for each question's spoken response.
- **EmotionRecord Collection:** Stores facial emotion analysis results captured from webcam frames at regular intervals during the session.
- **Report Collection:** Stores the final Confidence and Competency Report generated after each completed interview session.

4.9.3 Relationships Between Collections

- **One-to-Many (1:N):** One User may conduct many InterviewSessions.
- **One-to-Many (1:N):** One InterviewSession has many SpeechRecords (one per question) and many EmotionRecords (one per frame batch).
- **One-to-One (1:1):** One InterviewSession generates exactly one Report.
- **Many-to-One (N:1):** Many InterviewSessions may use the same JobRole.

4.9.4 Security and Data Integrity

To ensure confidentiality and integrity of stored data, the following measures are implemented:

- **Password Encryption:** Passwords are hashed using bcrypt before storage.
- **Role-Based Access Control (RBAC):** Admins and Candidates have distinct access privileges enforced at the API layer.
- **Input Validation:** All user inputs are validated both client-side and server-side before database operations.
- **HTTPS Communication:** All data transmission between client and server uses HTTPS encryption.

- **API Key Security:** Third-party API keys (Gemini, Whisper, ElevenLabs, DeepFace) are stored as server-side environment variables and never exposed to the client.
- **Database Backup:** MongoDB Atlas provides automated daily backups and point-in-time recovery.

4.9.5 Scalability Considerations

The database design incorporates scalability mechanisms to support future growth:

- **Horizontal Scaling:** MongoDB Atlas supports sharding for distributing large datasets across multiple nodes.
- **Indexing:** Indexes on user_id, session_id, and created_at improve query performance for dashboard and report retrieval.
- **Caching:** Frequently accessed data such as job role lists can be cached using Redis to reduce database load.
- **Load Balancing:** The Node.js backend can be deployed across multiple instances behind a load balancer for high availability.

4.10 Chapter Summary

This chapter presented the comprehensive software design of the Ghost Interview AI Mock Interviewer system. It translated the functional and non-functional requirements identified in Chapter 3 into structured architectural, behavioral, and data models.

The Object-Oriented Design methodology combined with a Layered Architecture was justified as the most suitable approach for this AI-integrated web application. The Agile Incremental process model was selected to accommodate iterative refinement of AI service integrations.

A high-level system overview was provided through the context diagram and three-tier architectural representation, clearly illustrating interactions between the React.js frontend, Node.js backend, four AI service APIs, and the MongoDB database.

Detailed UML models including activity diagrams for all three modules, class diagram, sequence diagrams for login and interview operations, and state transition diagrams were developed to describe system behavior and structure. Data flow diagrams at Level 1 and Level 2 illustrated how data moves across the system during interview sessions.

The chapter further elaborated the database design through an Entity Relationship Diagram, database schema diagram, and comprehensive data dictionaries for all five major collections. Security measures, scalability strategies, and data integrity considerations were also addressed.

Overall, this chapter establishes a clear and detailed technical blueprint that guides the implementation phase of the Ghost Interview system.

Chapter 5

Implementation

5.1 Introduction

Ghost Interview is an AI-powered mock-interview platform that gives job seekers a realistic, private environment in which to practise. A candidate selects a job role, then sits a live interview conducted by an AI avatar that asks each question aloud, captures the candidate's spoken answers through the microphone, and watches facial expressions through the webcam. When the session ends, the system analyses speech quality, filler-word usage, sentiment and non-verbal behaviour, and produces a Confidence and Competency Report with an overall readiness score, strengths and improvement areas that the candidate can export as a PDF.

This part reports the state of the project at Evaluation III. It presents the implemented modules together with their algorithms and screenshots captured from the running application (Chapter 5 / Implementation), and the software testing performed on those modules, including both positive and negative scenarios (Chapter 6 / Software Testing). The features described here represent approximately sixty percent of the overall project scope.

5.1.1 Project Objectives

- Provide a realistic AI-driven mock interview where an avatar speaks each question and the candidate answers by voice.
- Authenticate candidates and administrators securely with role-based access and hardened credential handling.
- Convert spoken answers to text and analyse filler words and sentiment to assess communication quality.
- Detect facial expressions in the browser during the interview to measure confidence, eye contact and nervousness.
- Generate a Confidence and Competency Report with a readiness score, strengths, improvement areas and one-click PDF export.
- Track improvement over time through an interview-history dashboard and platform-wide admin analytics.

5.1.2 Technology Stack

Table 5.1: Technology Stack of the Implemented System

Layer	Technology	Purpose
Frontend	React 19 + Vite	Single-page application UI and routing.
Styling	Tailwind CSS 4	Responsive, utility-first design system.
Backend	Node.js 20 + Express	REST API, session orchestration, business logic.
Database	MongoDB Atlas (Mongoose)	Persistent storage for users, sessions and reports.
Authentication	JWT + refresh cookie	Short-lived access token in memory; httpOnly refresh cookie.
AI — Questions	Google Gemini API	Interview question generation with curated fallback.
AI — Speech	OpenAI Whisper + Web Speech API	Speech-to-text with browser-native fallback.
AI — Voice	ElevenLabs API	AI avatar voice synthesis with browser TTS fallback.
AI — Emotion	face-api.js (browser)	In-browser facial expression detection; no video uploaded.
Analysis	sentiment (AFINN)	Lexicon-based sentiment scoring of transcripts.

5.2 Implementation (60% Modules Completion)

This chapter documents the modules that have been implemented and verified working at Evaluation III. For each module the functionality is described, the underlying algorithm is presented in a step-by-step algorithm table following the thesis template format, and the working application is demonstrated through screenshots captured from the live system.

5.2.1 User Registration Module

Functionality: A new candidate creates an account by providing a full name, a valid e-mail address and a password. The system validates each field, rejects duplicate e-mail addresses with a constant response to prevent account enumeration, hashes the password with bcrypt (cost factor 12), issues a JWT session, and dispatches a welcome e-mail (logged to the console in development when SMTP is not configured).

Algorithm 5.1: User Registration

Step	Description
1	User fills the registration form with full name, e-mail and password.
2	Validate input: name must be 2–100 characters; e-mail must match a valid e-mail format; password must meet the minimum strength requirement.
3	If any validation fails, return HTTP 400 with field-level messages and abort.
4	Query the users collection for an existing record with the same e-mail address.
5	If an account with that e-mail already exists, return HTTP 409 "An account with this email already exists" and abort.
6	Hash the password with bcrypt (cost factor 12) and create the user document with role defaulting to candidate and status active.
7	Send the welcome e-mail asynchronously (or log it when SMTP is not configured).
8	Return HTTP 201 with the safe user object (password hash and tokens excluded).

Screenshots

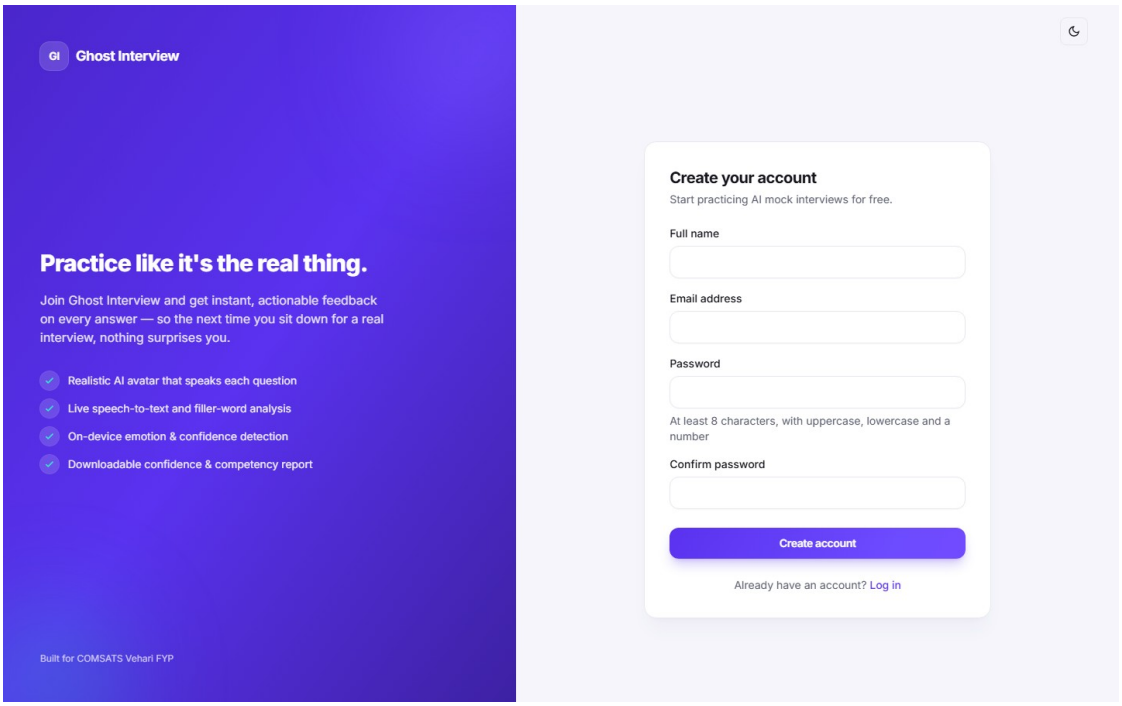


Figure 5.1: Registration form with name, e-mail and password fields.

5.2.2 User Login & Authentication Module

Functionality: A registered user signs in with e-mail and password. The server verifies the credentials against the stored bcrypt hash, enforces brute-force lockout after five failed attempts, blocks suspended accounts, and issues a short-lived JWT access token (kept in memory on the client) together with a long-lived refresh token in an httpOnly, Secure, SameSite=strict cookie. A refresh endpoint rotates both tokens, and logout-all invalidates every outstanding session by bumping the token version.

Algorithm 5.2: User Login & Authentication

Step	Description
1	User enters e-mail and password on the login form and submits.
2	Validate input: e-mail must be valid; password must not be empty. On failure return HTTP 400.
3	Fetch the user by e-mail, explicitly selecting the hidden passwordHash field.
4	If no active user is found, return HTTP 401 "Invalid email or password" (constant response to avoid user enumeration).
5	If the account is locked from repeated failures, return HTTP 429; if suspended, return HTTP 403.
6	Compare the submitted password with the stored bcrypt hash using a constant-time comparison.
7	On mismatch, increment failedLoginAttempts; after five failures set lockUntil for 15 minutes; return HTTP 401.
8	On success, reset the failed-attempt counter, update lastLoginAt, and save.
9	Issue a JWT access token and a refresh token stored in an httpOnly cookie; return HTTP 200 with the token and safe user.
10	The client keeps the access token in memory and redirects the user to the dashboard.

Screenshots

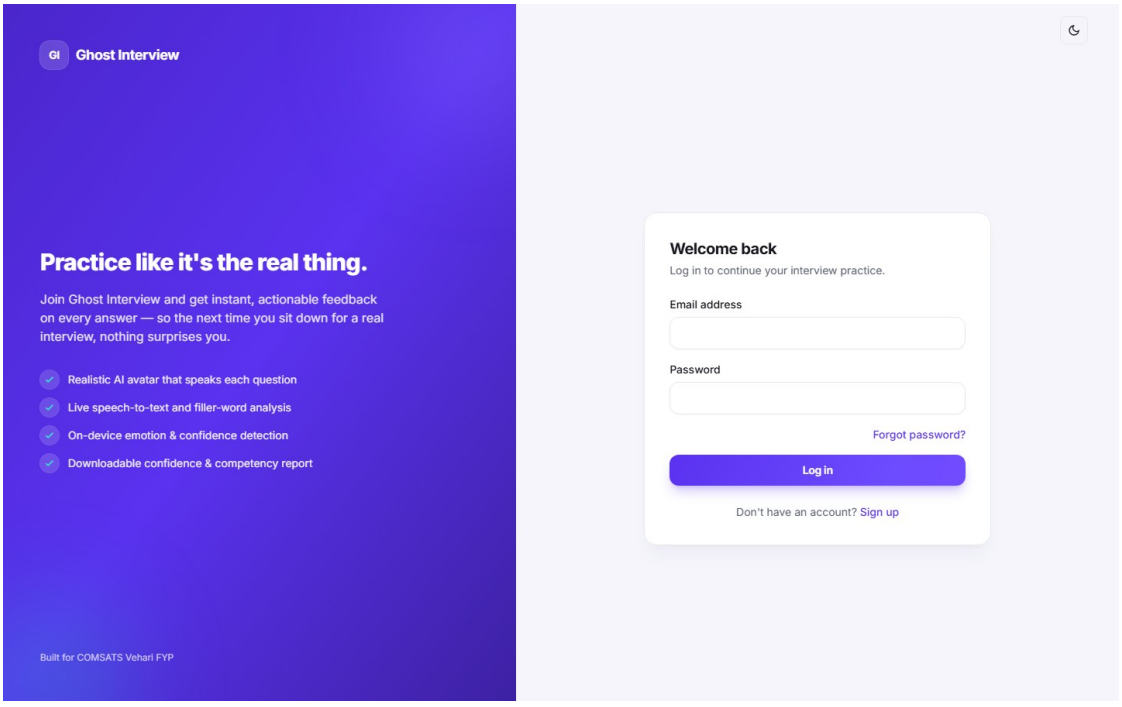


Figure 5.2: Login interface with e-mail and password fields.

5.2.3 Job Role Selection & Interview Initiation Module

Functionality: The candidate browses the active job roles and selects one to begin an interview. The server snapshots the questions onto the session — generating fresh questions via the Gemini API when configured, otherwise falling back to the role's curated question bank — so

that later edits to a role never change an in-progress or completed interview. A new InterviewSession is created with status active and the first question time-stamped.

Algorithm 5.3: Job Role Selection & Interview Initiation

Step	Description
1	Candidate opens the job-role selection screen; the client requests the active roles (GET /api/job-roles).
2	Validate the active job role exists and has at least one configured question; otherwise return HTTP 404/400.
3	Attempt to generate interview questions via the Gemini API for the selected role.
4	If Gemini is unavailable or returns too few questions, fall back to the role's curated question bank ordered by their sequence.
5	Snapshot the resolved question texts onto the session so later role edits cannot alter this interview.
6	Create the InterviewSession with status active, currentQuestionIndex 0, and the first question's askedAt time-stamp.
7	Return HTTP 201 with the session id, total question count and the first question.

Screenshots

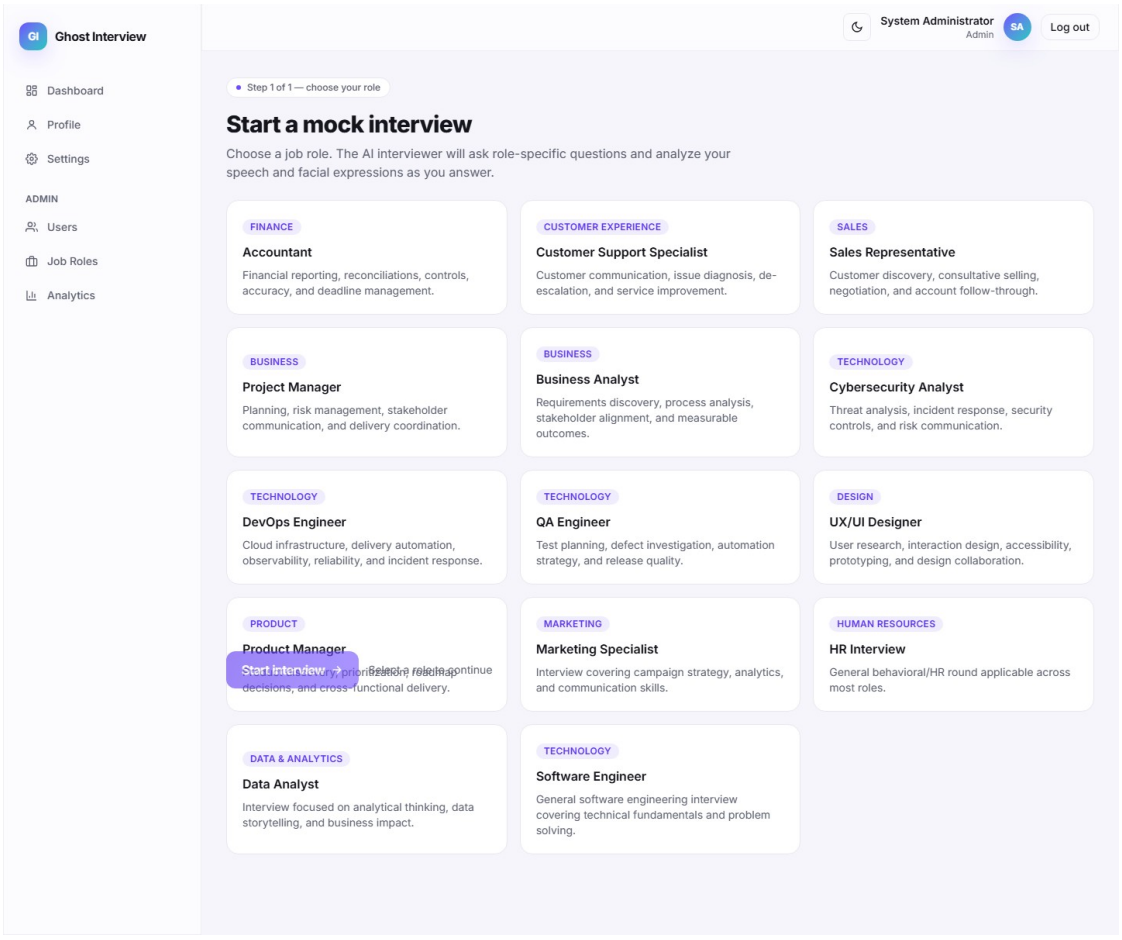


Figure 5.3: Job-role selection screen with available roles.

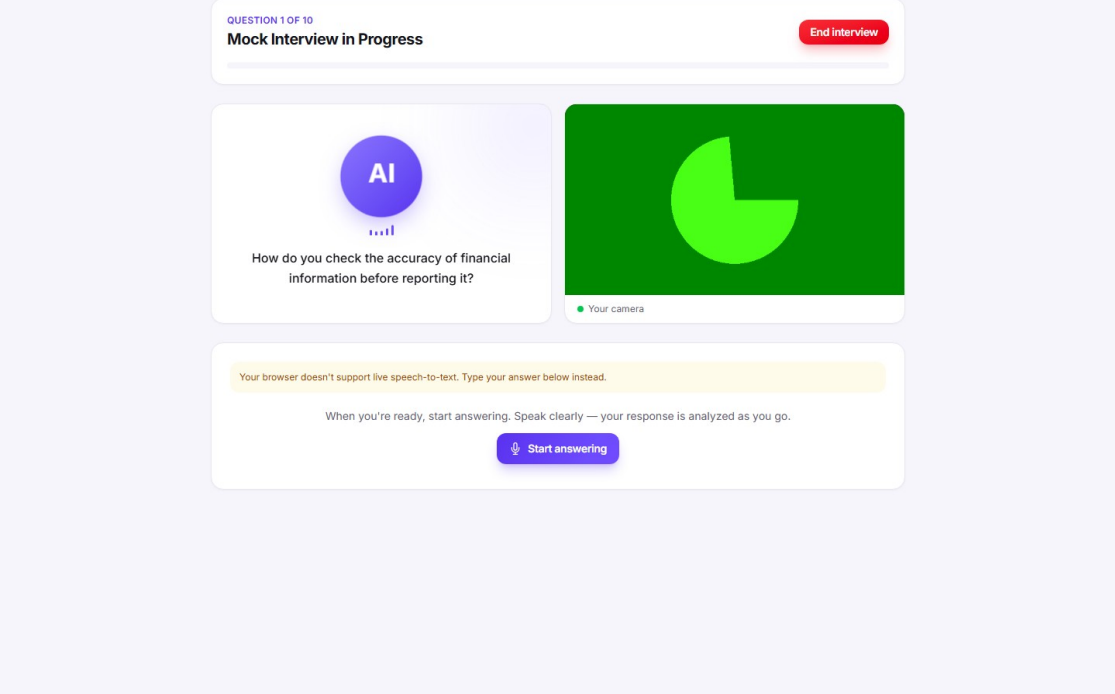


Figure 5.4: Interview initiated — first question presented to the candidate.

5.2.4 AI Interview Session Module

Functionality: During a live interview the AI avatar speaks each question (ElevenLabs voice when configured, otherwise the browser's speech synthesis), the candidate answers by voice with live speech-to-text (OpenAI Whisper when configured, otherwise the browser Web Speech API), and the webcam runs in-browser emotion detection throughout. Only microphone audio is uploaded when Whisper is configured; raw webcam video never leaves the browser.

Algorithm 5.4: AI Interview Session

Step	Description
1	Candidate starts the session; the client requests camera and microphone permission.
2	For the current question the client requests the spoken audio (GET /api/sessions/:id/voice); the server synthesises it with ElevenLabs, else the browser speaks it.
3	The candidate answers by voice; the browser captures audio and produces a live transcript (Web Speech API, or Whisper via POST /api/sessions/:id/transcribe when configured).
4	The client submits the transcript (POST /api/sessions/:id/responses).
5	Meanwhile the webcam stream is analysed in-browser with face-api.js and emotion readings are batched to POST /api/sessions/:id/emotion.
6	The server stores the SpeechRecord, advances to the next question, or sets status to processing on the last question.
7	When all questions are answered the candidate completes the session to trigger report generation.

Screenshots

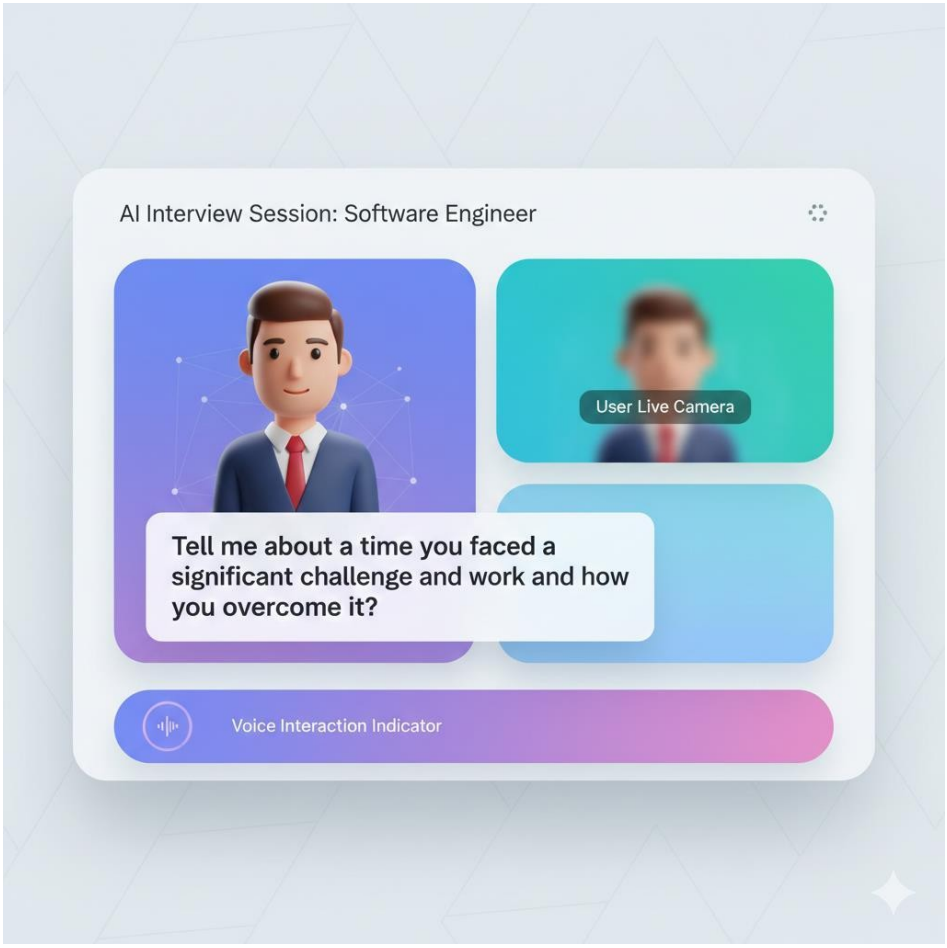


Figure 5.5: Live interview screen with AI avatar, question text and webcam feed.

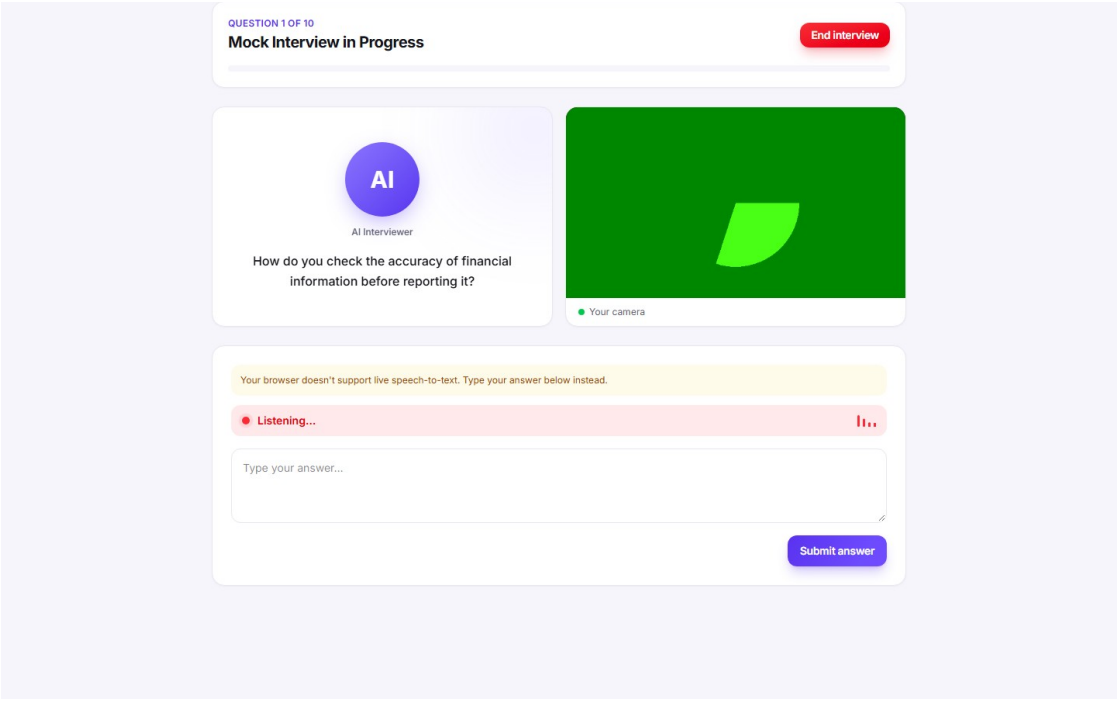


Figure 5.6: Live speech-to-text transcript of the candidate's spoken answer.

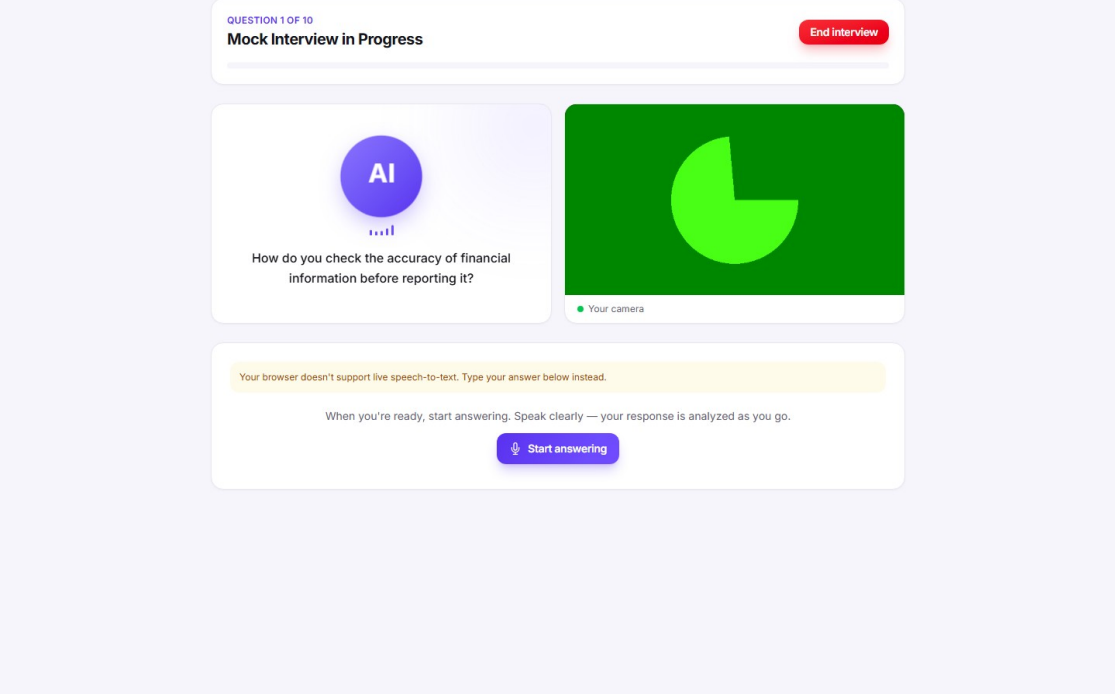


Figure 5.7: Browser-side emotion detection running over the webcam feed.

5.2.5 Speech-to-Text, Filler-Word & Sentiment Analysis Module

Functionality: Each submitted transcript is analysed in two deterministic passes. A word-boundary regular expression counts filler words and phrases (um, uh, like, you know, and so on), and the AFINN sentiment lexicon scores the emotional tone, clamped to a −1..1 range and labelled positive, negative or neutral. The results are stored on the SpeechRecord for later aggregation in the report.

Algorithm 5.5: Speech, Filler-Word & Sentiment Analysis

Step	Description
1	Receive the transcribed answer for the current question.
2	Lower-case the transcript and match it against a filler-word/phrase regex with word boundaries.
3	Count the total filler-word occurrences and record the distinct filler words used.
4	Run the AFINN sentiment analyzer to obtain a comparative score for the transcript.
5	Clamp the score to the −1..1 range and label it positive (> 0.1), negative (< −0.1) or neutral.
6	Persist a SpeechRecord with the transcript, fillerWordCount, fillerWords, sentimentScore and sentimentLabel.
7	Return the per-answer analysis to the client for immediate feedback.

Screenshots



Figure 5.8: Per-answer analysis showing filler-word count and sentiment.

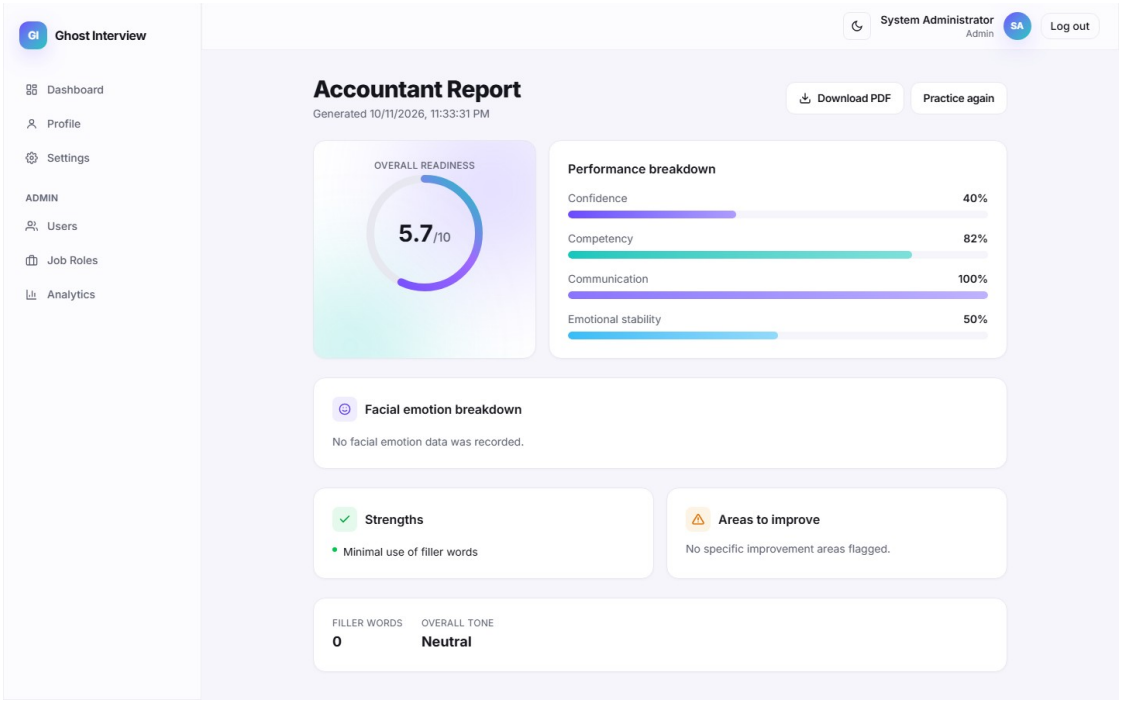


Figure 5.9: Filler words detected across a full interview session.

5.2.6 Facial Emotion Detection Module

Functionality: Facial expressions are analysed entirely in the browser using the pre-trained face-api.js model. For each sampled frame the model detects the dominant expression and derives eye-contact and nervousness indicators; these numeric readings — never the raw video — are batched and sent to the server, which stores them as EmotionRecord documents for the session.

Algorithm 5.6: Facial Emotion Detection

Step	Description
1	Load the pre-trained face-api.js models in the browser at session start.
2	Start a detection loop over the webcam stream, sampling frames at a fixed interval.
3	For each detected face, take the dominant expression as the emotion label.
4	Derive eye-contact and nervousness indicators from the detection results.
5	Batch the readings captured for the current question and POST them to /api/sessions/:id/emotion.
6	Validate the batch and bulk-insert the EmotionRecord documents linked to the session.
7	No raw webcam video is uploaded; only the derived numeric readings are stored.

Screenshots

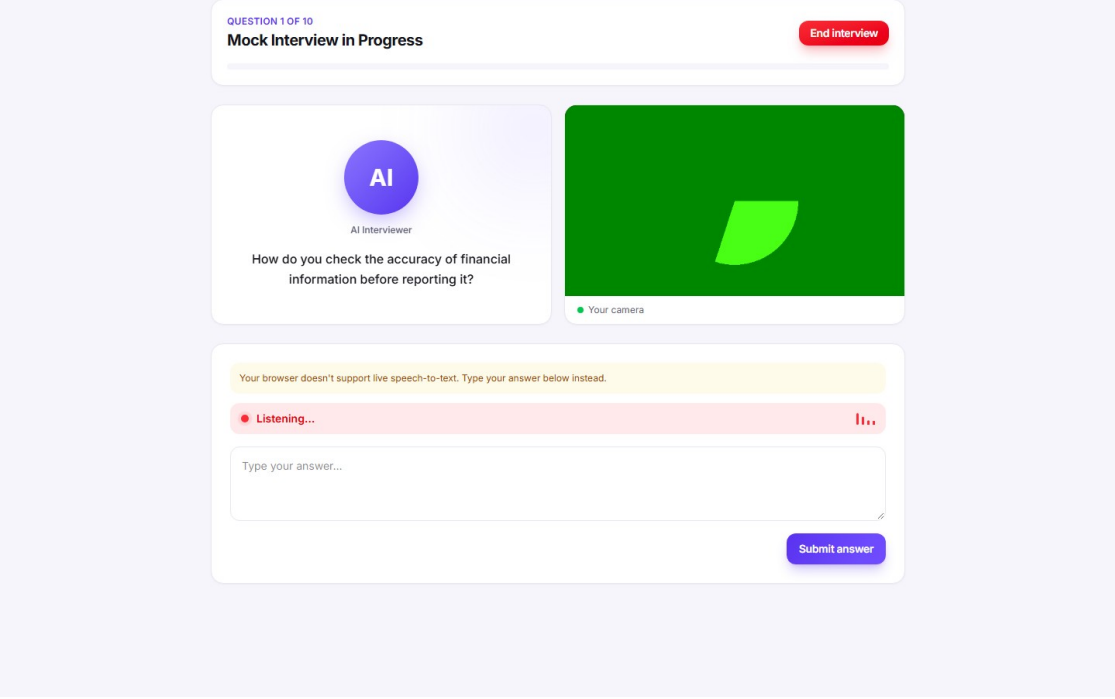


Figure 5.10: Real-time facial expression detection during the interview.

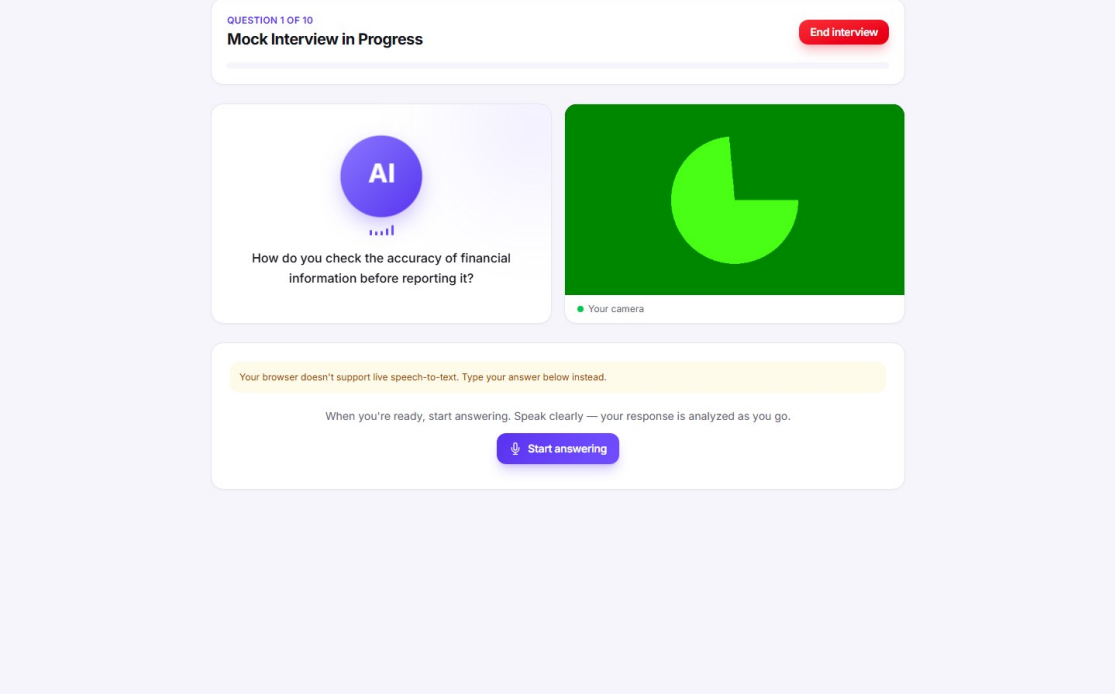


Figure 5.11: Batched emotion readings recorded for the session.

5.2.7 Confidence & Competency Report Module

Functionality: When a session is completed (or terminated early) the server loads all SpeechRecords and EmotionRecords and computes a deterministic Confidence and Competency Report: communication, confidence, emotional-stability and competency scores plus an overall readiness score out of 10. It also builds strengths and improvement areas and an emotion summary with a distribution and timeline. The report screen renders the scores and offers a one-click PDF export generated in the browser.

Algorithm 5.7: Confidence & Competency Report Generation

Step	Description
1	On complete/terminate, load the session's SpeechRecords (ordered) and EmotionRecords.
2	Compute answered ratio = answered questions / total questions.
3	Aggregate filler words across answers and compute fillerRatio = total filler words / total words.
4	Compute the positive-sentiment ratio and average sentiment across answers.
5	Build the emotion summary: dominant emotion, distribution percentages, average eye contact and nervousness, and a down-sampled timeline.
6	Derive communication = 100 − fillerRatio×400; confidence from eye contact, calmness and positive tone; emotional stability = (1 − nervousness)×100; competency as a weighted blend.
7	Compute overall readiness = average of confidence, competency and emotional stability, scaled to 0–10.
8	Assemble strengths and improvement areas from threshold rules on the computed metrics.
9	Upsert the Report, set the session status to completed, and return it; the client renders the scores and can export a PDF.

Screenshots

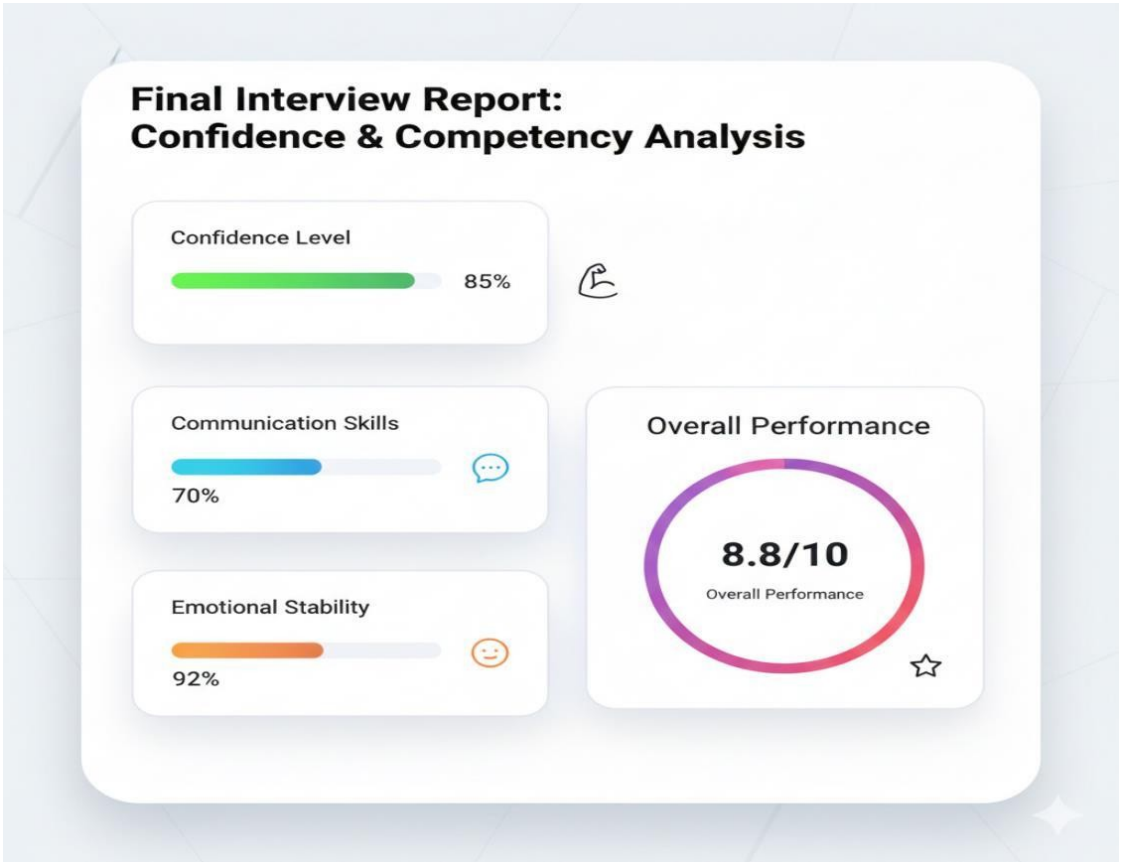


Figure 5.12: Confidence & Competency Report with overall readiness ring.

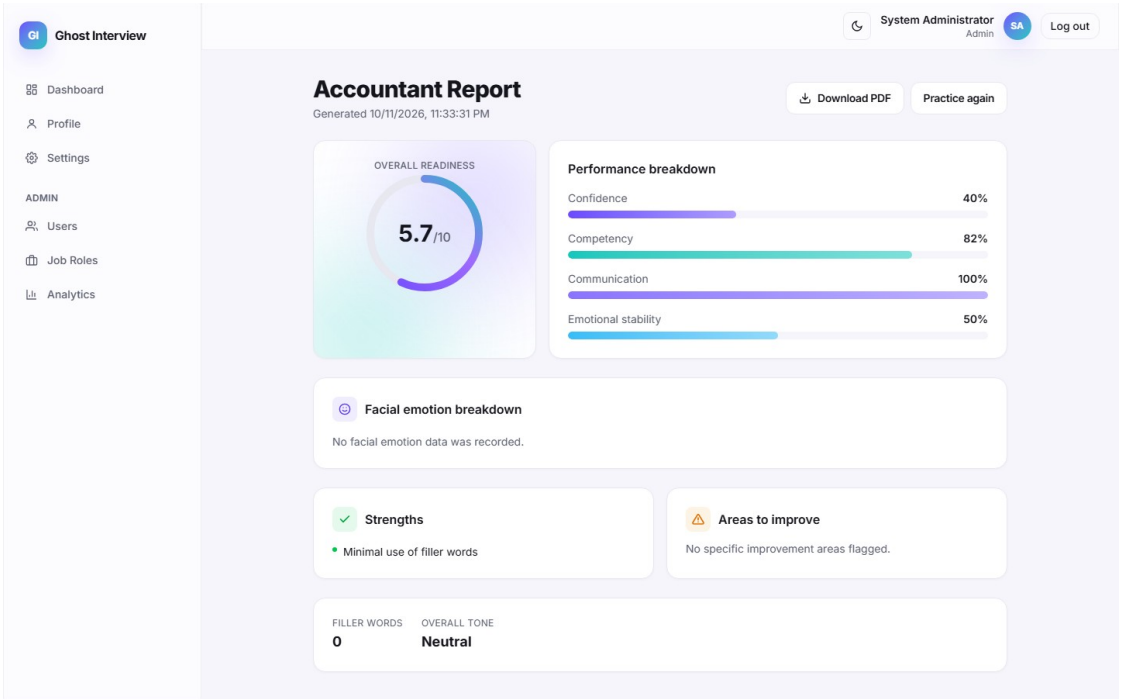


Figure 5.13: Performance breakdown — confidence, competency, communication and stability.

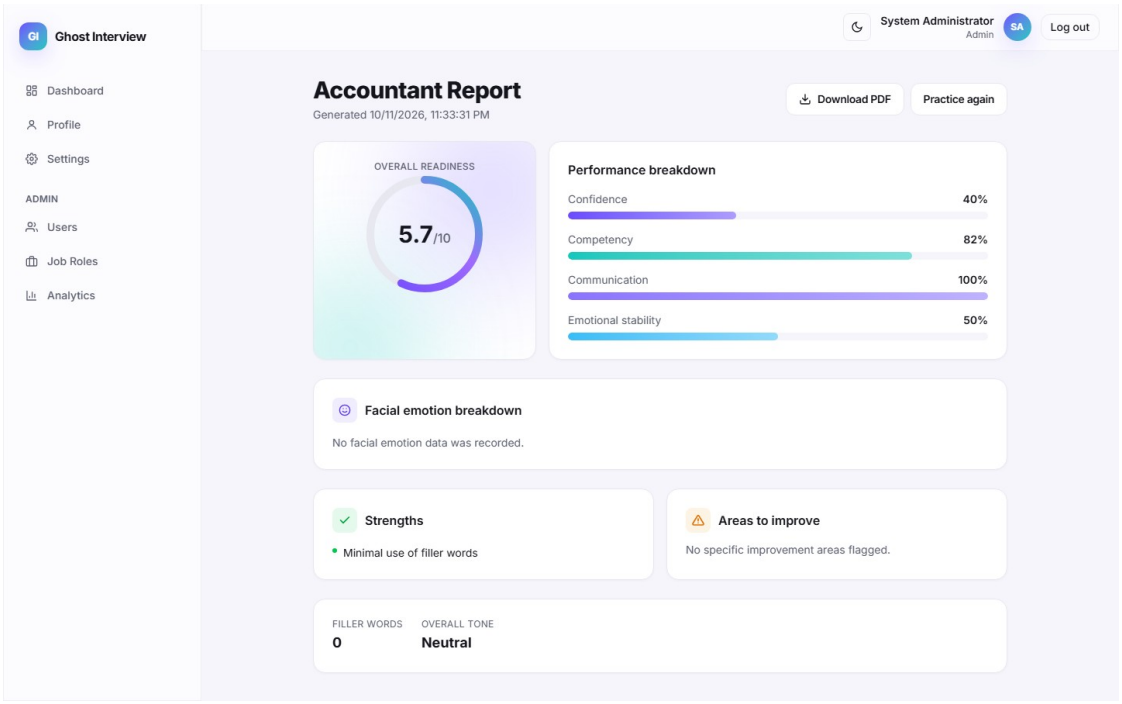


Figure 5.14: Emotion distribution and strengths / improvement areas.

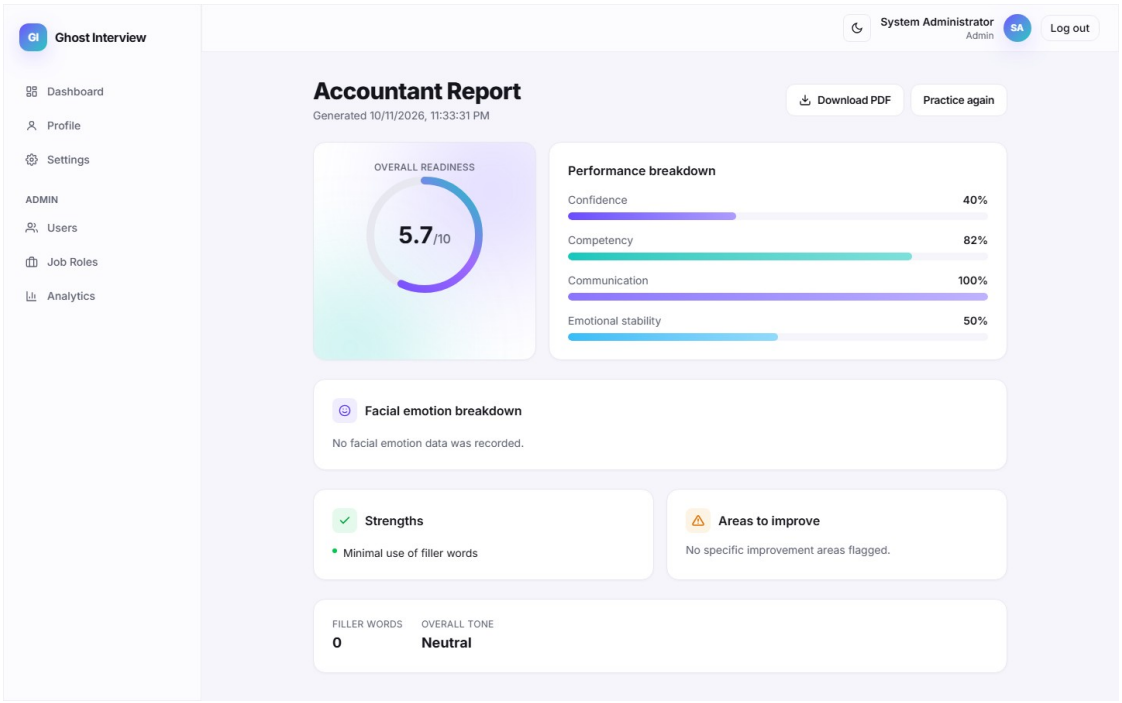


Figure 5.15: One-click PDF export of the report.

5.2.8 Interview History Dashboard Module

Functionality: The dashboard shows the candidate their interview history and past reports. Sessions and reports are returned paginated and sorted newest-first, scoped strictly to the authenticated user, with each report summarised by role, date and readiness so the candidate can track improvement over time.

Algorithm 5.8: Interview History Dashboard

Step	Description
1	Candidate opens the dashboard; the client requests sessions (GET /api/sessions) and reports (GET /api/reports).
2	Scope both queries to the authenticated user and apply pagination and newest-first sorting.
3	Return each session with its role, status, progress and times, and each report with its scores and generation date.
4	Render history cards linking to the full report for each completed interview.

Screenshots

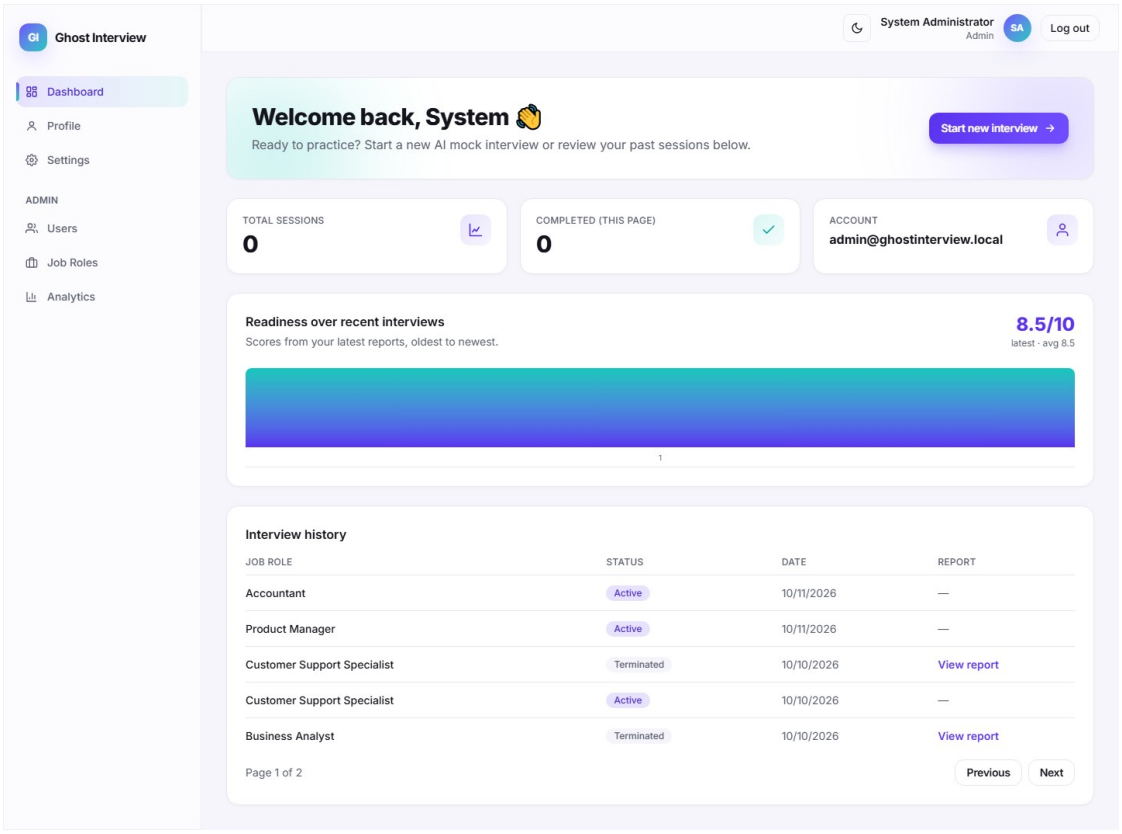


Figure 5.16: Dashboard showing interview history and past reports.

5.2.9 Admin Panel Module

Functionality: Administrators manage users and job roles and view platform-wide analytics. User management supports search, role/status filters, and suspend/reactivate or soft-delete, with guards preventing admins from affecting themselves or other admins and force-logging-out suspended users by bumping their token version. Job-role management supports create, edit and deactivate (soft delete). The analytics overview aggregates counts and average confidence, competency and readiness.

Algorithm 5.9: Admin Panel

Step	Description
1	Admin opens the admin panel; every admin route enforces role === 'admin'.
2	User management lists accounts with search, role and status filters; admins can suspend, reactivate or soft-delete candidates.
3	Reject attempts to suspend/delete other admins or the current admin; bump tokenVersion on suspend/delete to force logout.
4	Job-role management lets admins create, edit and deactivate roles and their question banks (soft delete preserves history).
5	The analytics endpoint aggregates platform-wide counts and average confidence, competency and readiness scores.

Screenshots

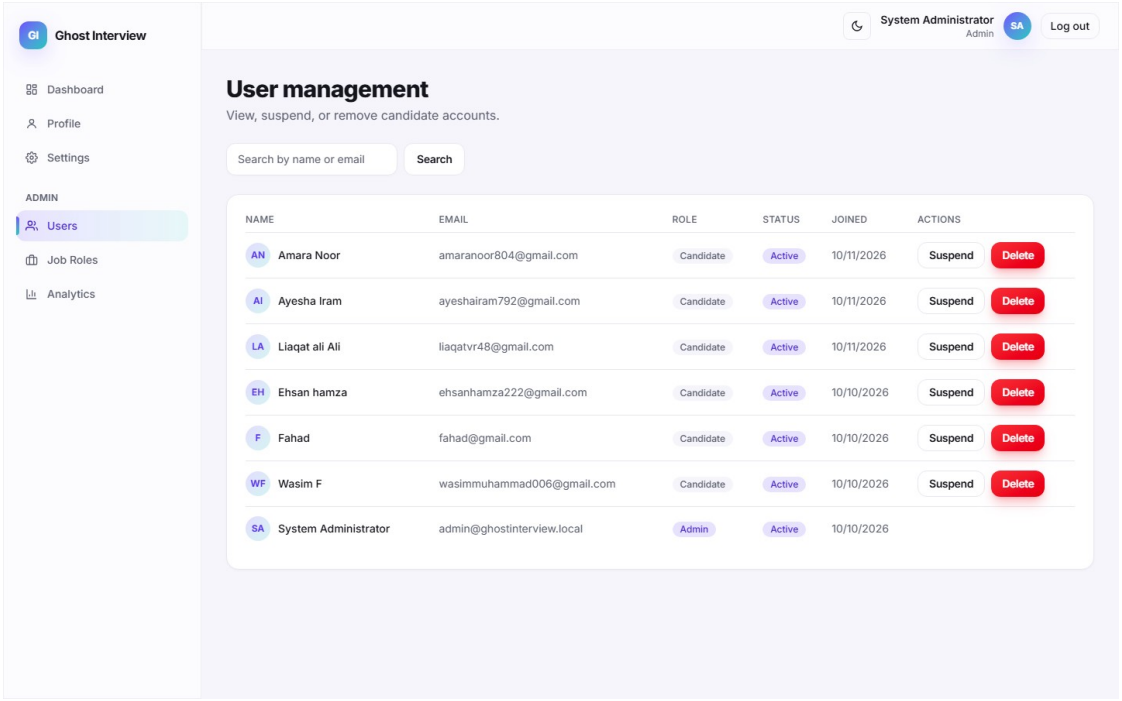


Figure 5.17: Admin user-management panel with search and status controls.

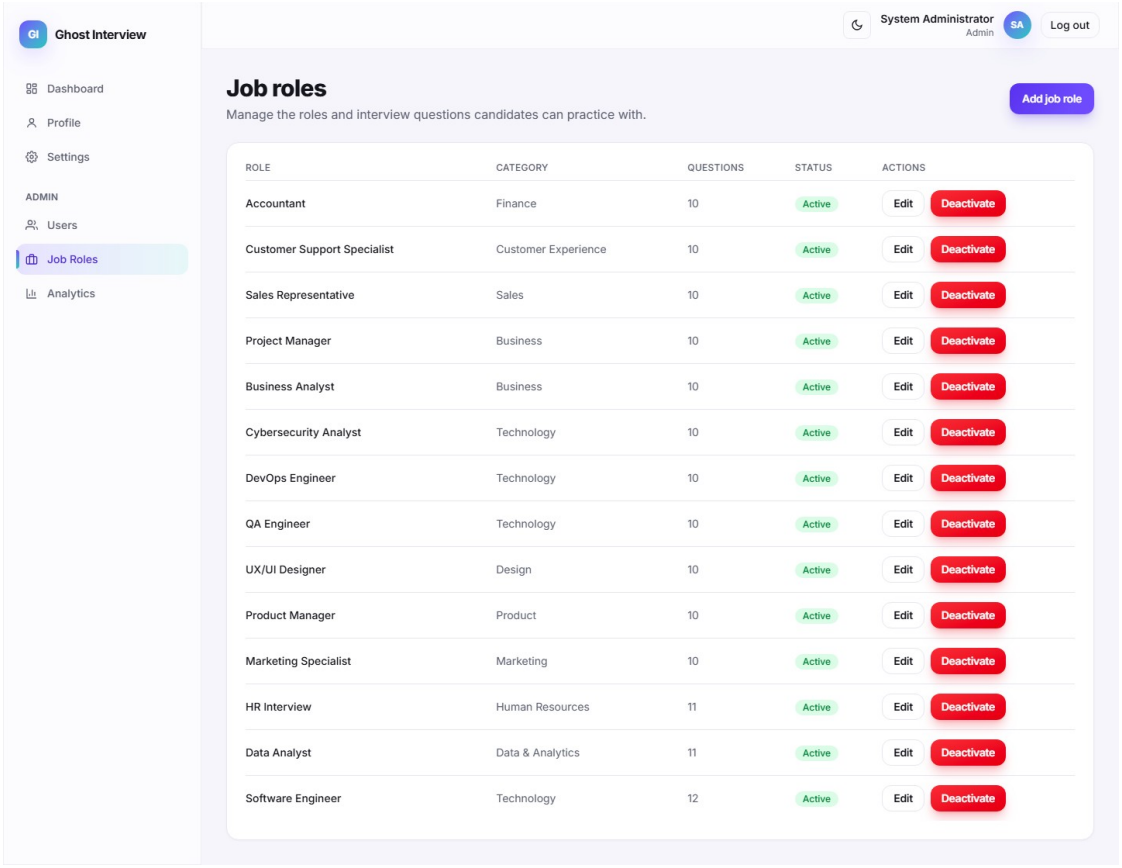


Figure 5.18: Admin job-role management screen.

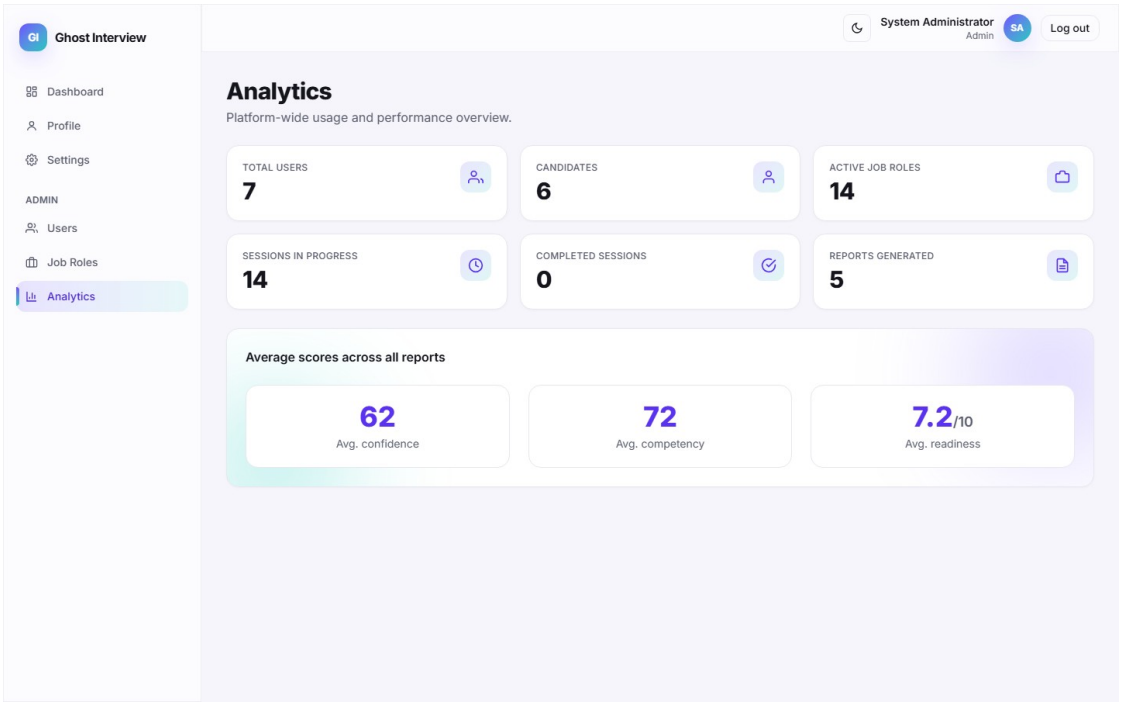


Figure 5.19: Platform-wide analytics overview.

5.2.10 Backend REST API & Security Module

Functionality: The Express backend exposes a documented REST API grouped under /auth, /job-roles, /sessions, /reports and /admin, with a /health probe. Every request is hardened with

Helmet security headers, request logging, Zod input validation, express-mongo-sanitize against NoSQL injection, rate limiting (tighter on auth routes), CSRF-guarded refresh, JWT authentication and per-resource ownership checks.

5.3 Module Completion Summary

Table 5.2: Module Completion Summary at Evaluation III

Module	Status	Key Capability
User Registration	Completed	Validation, duplicate check, bcrypt hashing, welcome e-mail.
User Login & Authentication	Completed	Credential verification, brute-force lockout, JWT + refresh tokens.
Job Role Selection & Initiation	Completed	Active-role listing, Gemini/curated question snapshot.
AI Interview Session	Completed	AI avatar voice, live speech-to-text, webcam emotion capture.
Speech, Filler-Word & Sentiment	Completed	Filler-word detection and AFINN sentiment per answer.
Facial Emotion Detection	Completed	In-browser face-api.js detection; only readings stored.
Confidence & Competency Report	Completed	Deterministic scoring, PDF export, strengths/improvements.
Interview History Dashboard	Completed	Per-user paginated session and report history.
Admin Panel	Completed	User & job-role management, platform analytics.
Backend REST API & Security	Completed	Helmet, Zod, sanitization, rate limiting, RBAC, ownership.
Hume / DeepFace Server Integration	In Progress	Specified provider not yet integrated; browser model used.
Production Deployment Validation	In Progress	HTTPS deploy, load and real-device testing pending.

Chapter 6

Software Testing

6.1 Introduction

This chapter documents the software testing carried out on the completed modules. Each module is covered by test cases that specify a Test Case ID, scenario, input data, expected result, actual result and a Pass/Fail status. Both positive (valid) and negative (invalid) scenarios are included, and screenshots captured from the live system provide evidence of the actual results.

6.2 Authentication Module Testing

The registration and login flows were tested with valid credentials, an incorrect password, empty fields, an unregistered e-mail, a duplicate e-mail, and repeated failed logins to confirm the brute-force lockout.

Table 6.1: Test Cases – Authentication Module

TC ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-L1	Login with valid credentials	candidate@ghost.demo / Password123!	Successful login, JWT issued, redirect to dashboard	Logged in, redirected to dashboard	**PASS**
TC-L2	Login with incorrect password	candidate@ghost.demo / WrongPass1	Error: Invalid email or password	Error banner shown, no token issued	**PASS**
TC-L3	Login with empty fields	(submit empty form)	Required-field validation	Validation messages displayed	**PASS**
TC-L4	Login with unregistered e-mail	ghost@ghost.demo / Password123!	Invalid email or password	Authentication failure shown	**PASS**
TC-L5	Repeated failed logins	5 wrong-password attempts	Account locked (HTTP 429) after 5 tries	Lockout message shown	**PASS**
TC-R1	Register with valid information	New name, unique e-mail, strong password	Account created, welcome e-mail, success shown	Account created, logged in	**PASS**
TC-R2	Register with existing e-mail	candidate@ghost.demo (already exists)	Duplicate e-mail error (409)	"An account with this email already exists"	**PASS**
TC-R3	Register with invalid input	Bad e-mail format, short password	Field-level validation errors (400)	Validation messages displayed	**PASS**

Screenshots

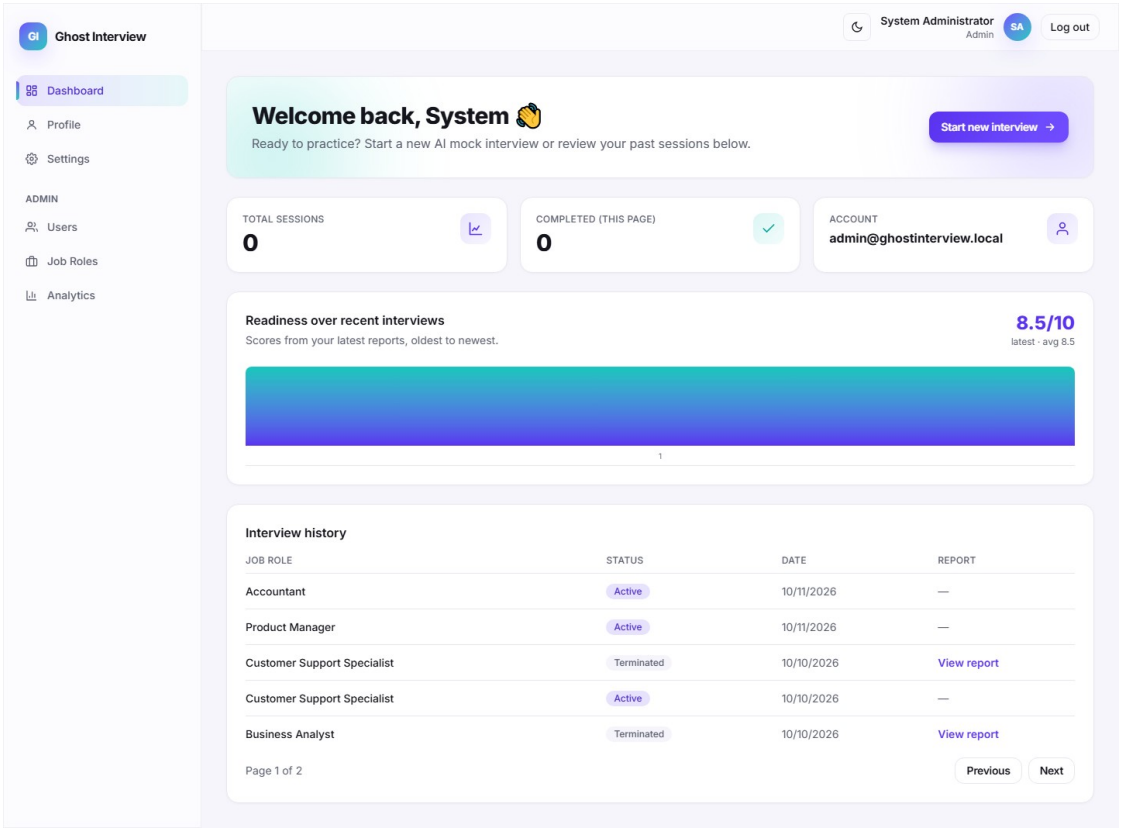


Figure 6.1: TC-L1 — successful login and redirection.

6.3 Interview Workflow & AI Module Testing

The interview workflow was tested end-to-end: starting a session, submitting answers, batching emotion readings, and completing the session. Speech and sentiment analysis, the AI providers' fallback behaviour, and permission handling were verified against the live system.

Table 6.2: Test Cases – Interview Workflow and AI Modules

TC ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-I1	Start interview session	Select active job role	Session created, first question shown	Session active, question displayed	**PASS**
TC-I2	Submit answer & advance	Spoken answer for Q1	Analysis returned, next question shown	Filler/sentiment analysis + next question	**PASS**
TC-I3	Record emotion batch	face-api.js readings for Q1	Readings stored for session	"Recorded N emotion reading(s)"	**PASS**
TC-I4	Complete session & report	All questions answered	Confidence & competency report generated	Report with scores and insights shown	**PASS**
TC-I5	Deny camera/mic permission	Block browser permissions	Graceful permission error, session halts	Permission error shown, no crash	**PASS**
TC-S1	Filler-word detection	Transcript with um / uh / like	Filler words counted	Correct count and list returned	**PASS**
TC-S2	Sentiment scoring	Positive and negative transcripts	Correct positive/neutral/negative label	Labels match transcript tone	**PASS**

Screenshots

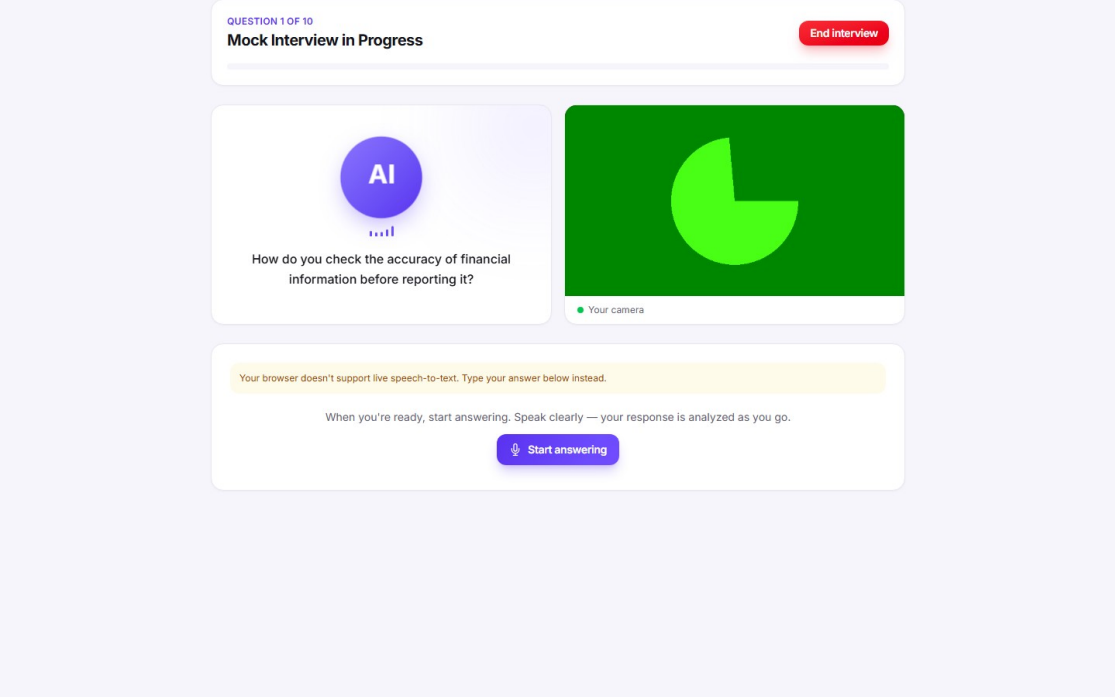


Figure 6.2: TC-I1 — interview session started with first question.

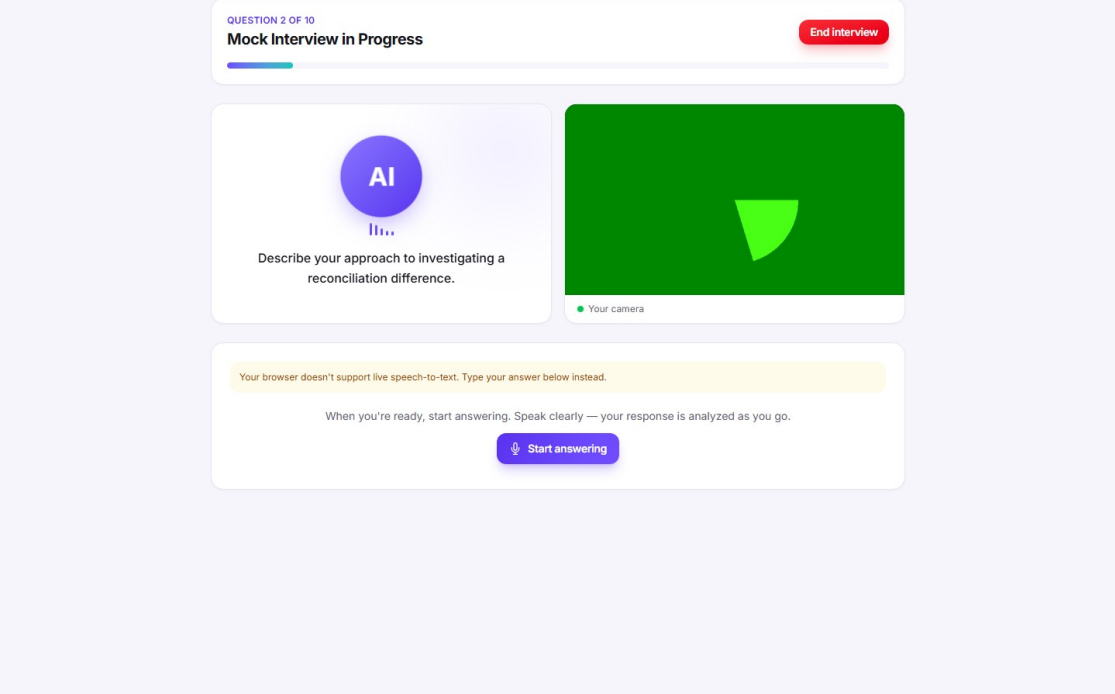


Figure 6.3: TC-I2 — answer submitted with per-answer analysis.

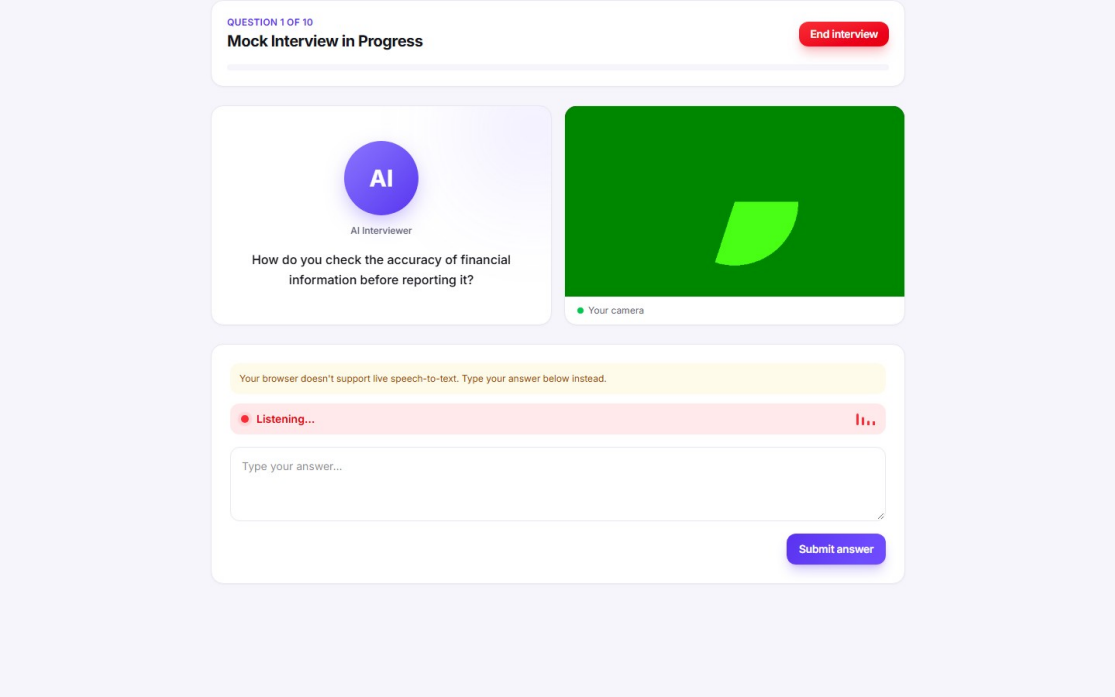


Figure 6.4: TC-I3 — emotion batch recorded during the session.

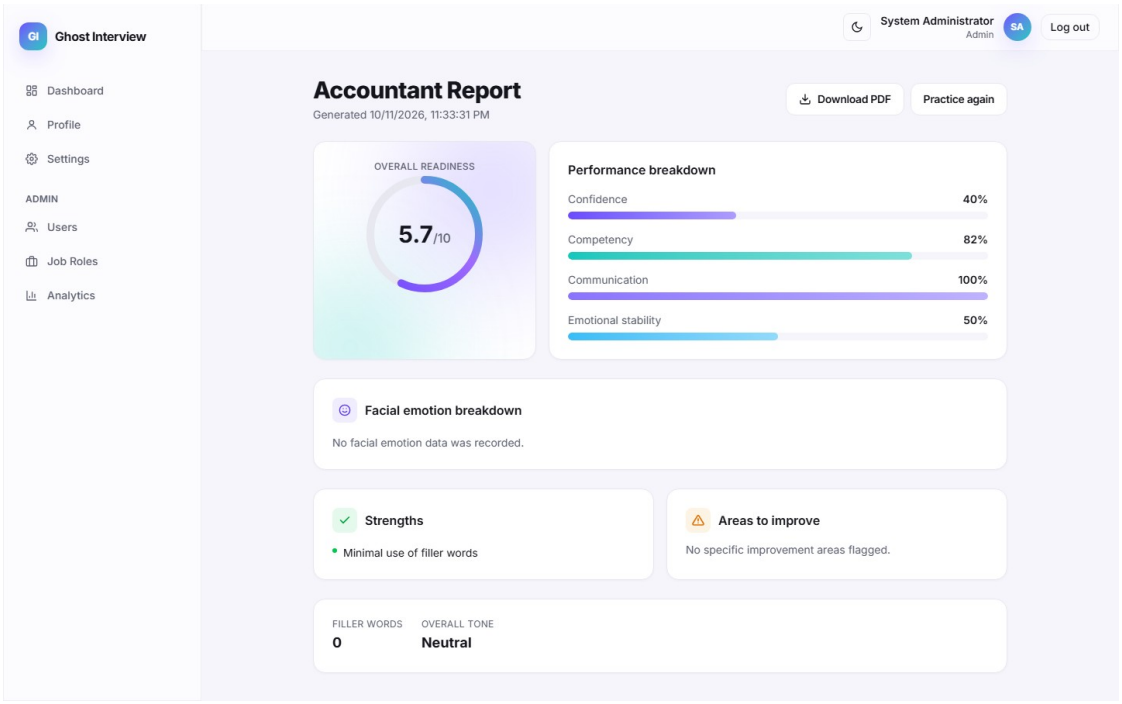


Figure 6.5: TC-I4 — report generated after completing the session.

6.4 Reporting, Dashboard & API Testing

The remaining completed modules were tested against the live system to confirm correct data retrieval, rendering, ownership enforcement and API responses, including PDF export and admin analytics.

Table 6.3: Test Cases – Reporting, Dashboard and API

TC ID	Scenario	Input Data	Expected Result	Actual Result	Status
TC-P1	View report & export PDF	Completed session id	Report rendered, PDF downloaded	Report shown, PDF downloaded	**PASS**
TC-P2	Partial report on early exit	Terminate after 1 answer	Partial report with isPartial = true	Partial report shown with notice	**PASS**
TC-D1	List interview history	GET /api/sessions & /api/reports	Paginated history returned	History dashboard populated	**PASS**
TC-O1	Ownership check on report	Another user's session id	Forbidden (403)	"This report does not belong to you"	**PASS**
TC-H1	API health check	GET /api/health	status = ok	{"success":true,...} returned	**PASS**
TC-A1	Admin analytics overview	GET /api/admin/analytics (admin)	Platform counts + average scores	Analytics dashboard populated	**PASS**
TC-A2	RBAC blocks non-admin	GET /api/admin/users (candidate token)	Forbidden (403)	Access denied	**PASS**

Screenshots

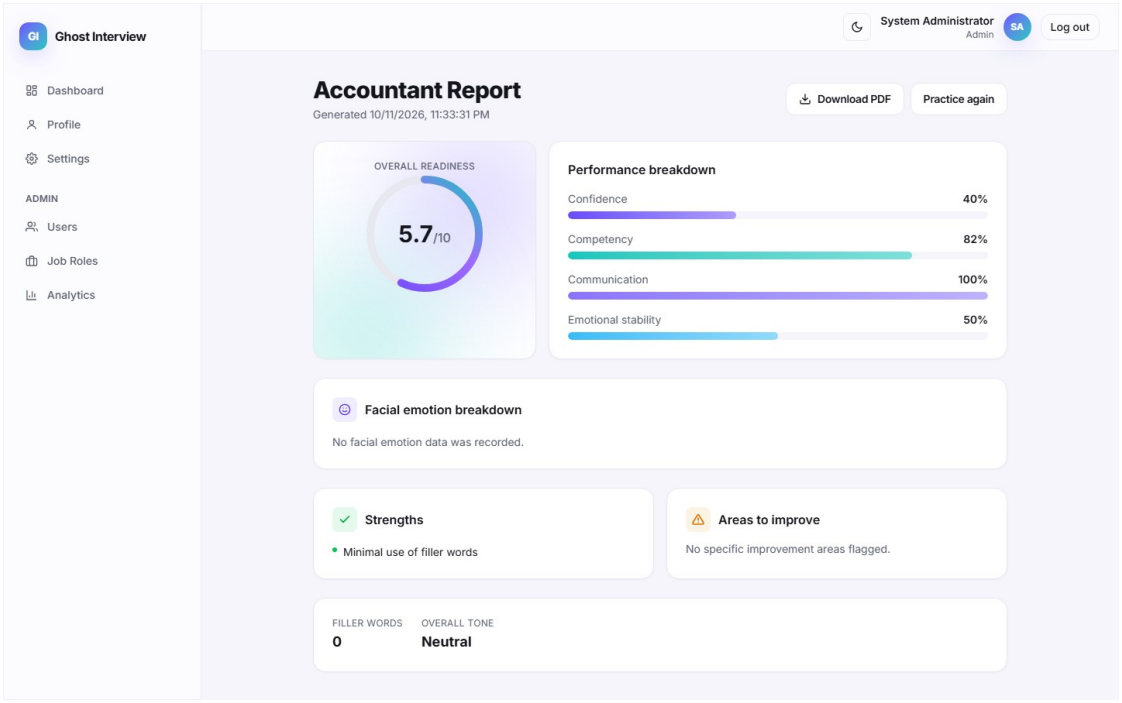


Figure 6.6: TC-P1 — Confidence & Competency Report with PDF export.

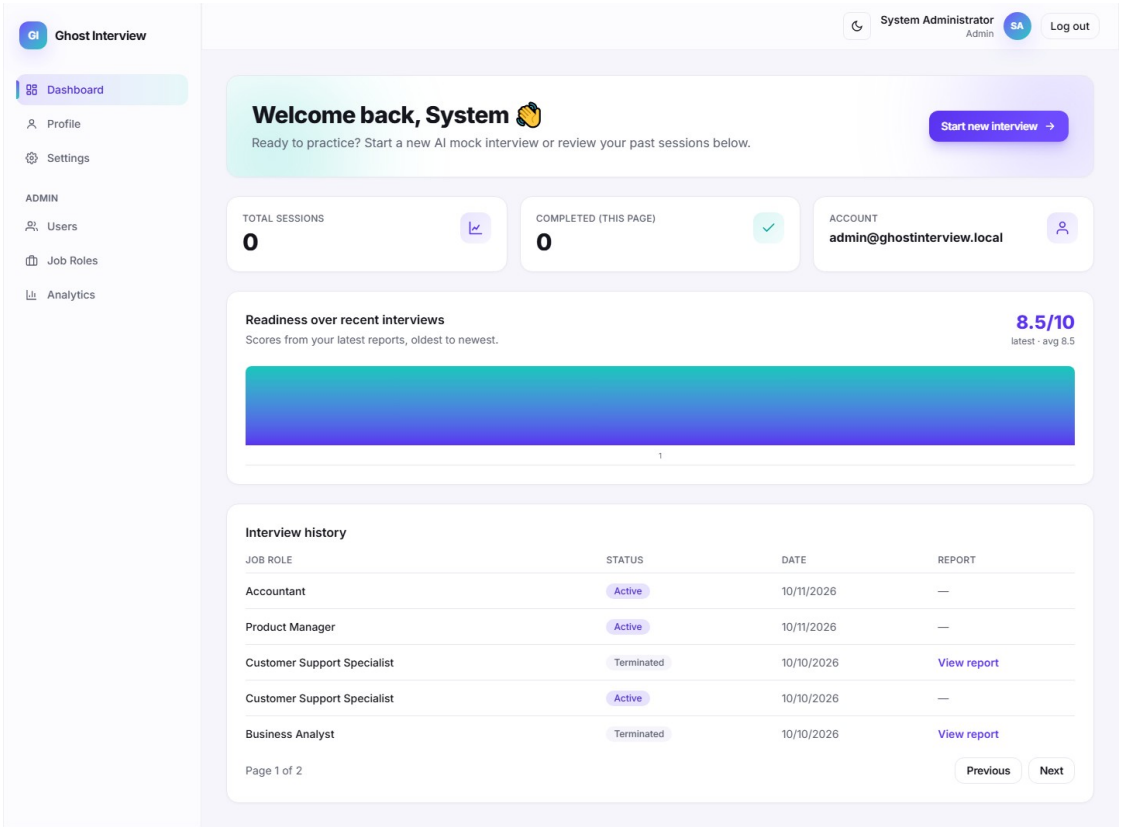


Figure 6.7: TC-D1 — interview history dashboard.

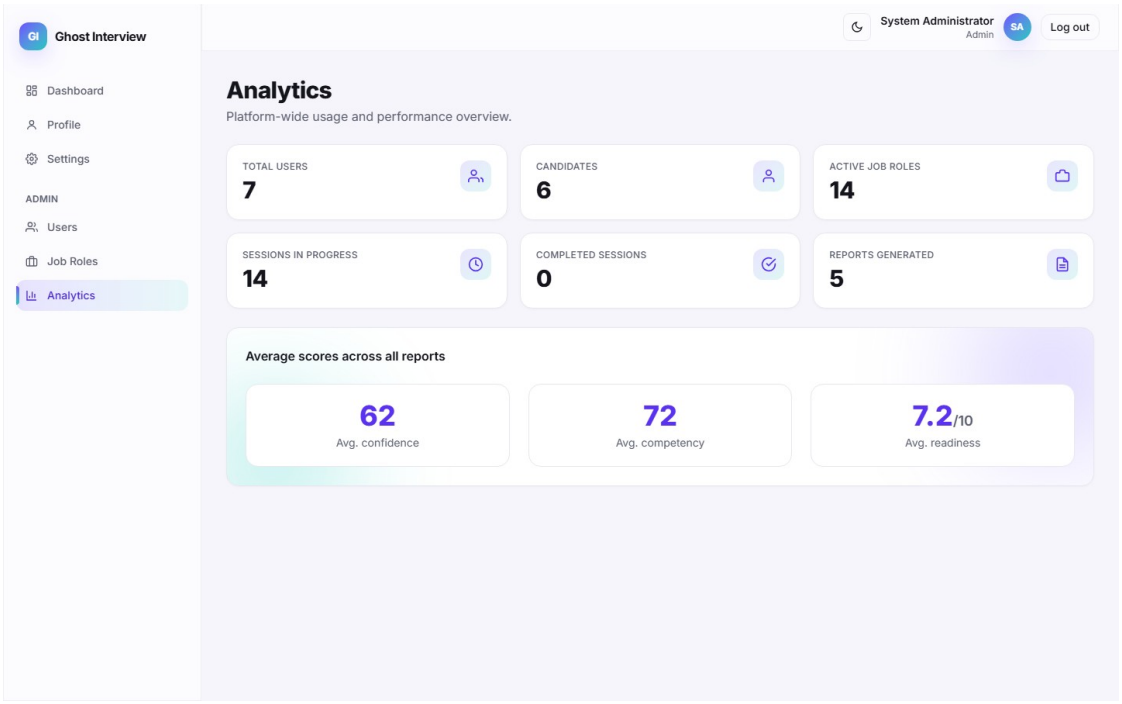


Figure 6.8: TC-A1 — admin analytics overview.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This document has presented the implemented modules of the Ghost Interview AI Mock Interviewer, together with their algorithms and screenshots captured from the live application (Chapter 5 / Implementation), and the software testing performed on those modules with both positive and negative scenarios (Chapter 6 / Software Testing). Every documented module is supported by a technical explanation, a step-by-step algorithm table, and evidence from the running system.

7.2 Future Work

The features documented in this report represent approximately sixty percent of the overall project. The remaining work focuses on integrating the specified Hume/DeepFace server-side emotion provider, completing production HTTPS deployment with load and real-device camera/microphone acceptance testing, and extending analytics — all of which build directly on the working modules demonstrated and tested in this report.

References

[1] A. Radford et al., “Robust Speech Recognition via Large-Scale Weak Supervision,” OpenAI, 2022. [Online]. Available: https://openai.com/research/whisper (Accessed: Jan. 2026).

[2] I. Goodfellow, Y. Bengio, and A. Courville, Deep Learning. Cambridge, MA, USA: MIT Press, 2016.

[3] S. Li and W. Deng, “Deep Facial Expression Recognition: A Survey,” IEEE Transactions on Affective Computing, vol. 13, no. 3, pp. 1195–1215, 2022.

[4] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, Minneapolis, MN, USA, 2019, pp. 4171–4186.